

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

Oleh:

Noor Khalisa

NIM. 2410817220012

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Noor Khalisa
NIM : 2410817220012

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL	5
SOAL 1	6
B. Source Code	7
C. Output Program.....	38
D. Pembahasan.....	41
SOAL 2	72
A. Pembahasan.....	72
TAUTAN GIT.....	73

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose	39
Gambar 2. Screenshot Hasil Jawaban Soal 1 XML	41

DAFTAR TABEL

Tabel 1. Source Song.kt (Compose)	7
Tabel 2. Source Song.kt (Compose)	7
Tabel 3. Source Code MainActivity.kt (Compose) Baru	9
Tabel 4. Source Code Song.kt (XML)	17
Tabel 5. Source Code SongData.kt (XML).....	17
Tabel 6. Source Code strings.xml eng (XML).....	19
Tabel 7. Source Code strings.xml id (XML)	20
Tabel 8. Source Code item_song.xml (XML).....	21
Tabel 9. Source Code fragment_home.xml (XML)	23
Tabel 10. Source Code fragment_detail.xml (XML).....	25
Tabel 11. Source Code fragment_language.xml (XML)	26
Tabel 12. Source Code SongAdapter.kt (XML)	27
Tabel 13. Source Code HomeFragment.kt (XML)	28
Tabel 14. Source Code DetailFragment.kt (XML)	31
Tabel 15. Source Code LanguageFragment.kt (XML)	32
Tabel 16. Source Code MainActivity.kt (XML).....	33
Tabel 17. Source Code Tambahan MusicApplication.kt (Compose)	34
Tabel 18. Source Code Tambahan SongViewModel.kt (Compose)	34
Tabel 19. Source Code Tambahan MusicApplication.kt (XML).....	35
Tabel 20. Source Code Tambahan SongViewModel.kt (XML)	36

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding
9. Aplikasi Mendukung Multi-language (bahasa indonesia dan inggris)
10. Terdapat 1 halaman/screen untuk melakukan pergantian Bahasa
11. Gunakan Icon dari library 'material' (material,material2,material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
12. Diatas Scrollable list, terdapat sebuah "Item Highlight" yang menggunakan Carousel atau LazyRow dengan ketentuan:
 - Item bisa bergeser secara finite atau infinite loop sebanyak jumlah item yang ada pada scrollable-list

B. Source Code

Tabel 1. Source Song.kt (Compose)

1	package com.example.modul3_compose_baru
2	
3	import java.io.Serializable
4	
5	data class Song(6 val id: Int, 7 val title: String, 8 val albumName: String, 9 val year: String, 10 val imageResId: Int, 11 val externalLink: String, 12 val descriptionResId: Int 13) : Serializable

Tabel 2. Source Song.kt (Compose)

1	package com.example.modul3_compose_baru
2	
3	object SongData { 4 val songs = listOf(5 Song(6 id = 1, 7 title = "Nothing's Gonna Hurt You Baby", 8 albumName = "I.", 9 year = "2012", 10 imageResId = R.drawable.album_i, 11 externalLink = "https://open.spotify.com/track/3W7KHojYGgYaoX9ogKO9hU?s i=bc234cdf897d49b3", 12 descriptionResId = R.string.desc_i 13), 14 Song(15 id = 2, 16 title = "Flash", 17 albumName = "Cigarettes After Sex", 18 year = "2017", 19 imageResId = R.drawable.album_cas, 20 externalLink = "https://open.spotify.com/track/6eknHC7HjAKchl23x2A7dW?s i=7ed322498ba447be", 21 descriptionResId = R.string.desc_flash 22), 23 Song(24 id = 3, 25 title = "Opera House",

```

26         albumName = "Cigarettes After Sex",
27         year = "2017",
28         imageResId = R.drawable.album_cas,
29         externalLink =
"https://open.spotify.com/track/2Ddfm2NQwTRsf7YVlt258S?si=f70b90493a034663",
30         descriptionResId = R.string.desc_opera
31     ),
32     Song(
33         id = 4,
34         title = "Sweet",
35         albumName = "Cigarettes After Sex",
36         year = "2017",
37         imageResId = R.drawable.album_cas,
38         externalLink =
"https://open.spotify.com/track/2KhrPRV0V1FS2l4eQMJUWt?si=77d6282cd2044d80",
39         descriptionResId = R.string.desc_sweet
40     ),
41     Song(
42         id = 5,
43         title = "Sesame Syrup",
44         albumName = "Crush",
45         year = "2018",
46         imageResId = R.drawable.album_crush,
47         externalLink =
"https://open.spotify.com/track/6GcZ4pdtZgbf2ptgHG31bq?si=953557105c374872",
48         descriptionResId = R.string.desc_sesame
49     ),
50     Song(
51         id = 6,
52         title = "Neon Moon",
53         albumName = "Neon Moon",
54         year = "2018",
55         imageResId = R.drawable.album_neon_moon,
56         externalLink =
"https://open.spotify.com/track/5r7QVaDpYeWMcbxxrkqb6H?si=489aa904e83f47f1",
57         descriptionResId = R.string.desc_neon
58     ),
59     Song(
60         id = 7,
61         title = "Cry",
62         albumName = "Cry",
63         year = "2019",
64         imageResId = R.drawable.album_cry,

```

65	externalLink	=
	"https://open.spotify.com/track/7mDTvYD2ieE4Q28XFziMfJ?s	
	i=89deal8975c047e9",	
66	descriptionResId = R.string.desc_cry	
67	)	
68	)	
69	}	

Tabel 3. Source Code MainActivity.kt (Compose) Baru

1	package com.example.modul3_compose_baru
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import androidx.activity.ComponentActivity
7	import androidx.activity.compose.setContent
8	import androidx.appcompat.app.AppCompatActivity
9	import androidx.appcompat.app.AppCompatActivity
10	import androidx.compose.foundation.Image
11	import androidx.compose.foundation.background
12	import androidx.compose.foundation.layout.*
13	import androidx.compose.foundation.layout.Arrangement
14	import androidx.compose.foundation.lazy.LazyColumn
15	import androidx.compose.foundation.lazy.LazyRow
16	import androidx.compose.foundation.lazy.items
17	import androidx.compose.foundation.rememberScrollState
18	import
	androidx.compose.foundation.shape.RoundedCornerShape
19	import androidx.compose.foundation.verticalScroll
20	import androidx.compose.material3.*
21	import androidx.compose.runtime.Composable
22	import androidx.compose.runtime.LaunchedEffect
23	import androidx.compose.runtime.collectAsState
24	import androidx.compose.runtime.getValue
25	import androidx.compose.ui.Alignment
26	import androidx.compose.ui.Modifier
27	import androidx.compose.ui.draw.clip
28	import androidx.compose.ui.graphics.Color
29	import androidx.compose.ui.layout.ContentScale
30	import androidx.compose.ui.platform.LocalContext
31	import androidx.compose.ui.res.painterResource
32	import androidx.compose.ui.res.stringResource
33	import androidx.compose.ui.text.font.FontWeight
34	import androidx.compose.ui.unit.dp
35	import androidx.compose.ui.unit.sp

```

36 import androidx.core.os.LocaleListCompat
37 import androidx.lifecycle.viewmodel.compose.viewModel
38 import androidx.navigation.NavController
39 import androidx.navigation.NavType
40 import androidx.navigation.compose.NavHost
41 import androidx.navigation.compose.composable
42 import androidx.navigation.compose.rememberNavController
43 import androidx.navigation.navArgument
44
45 val LavenderBlush = Color(0xFFFFFE5EC)
46 val PinkChampagne = Color(0xFFFFFC2D1)
47 val Watermelon = Color(0xFFFB6F92)
48
49 class MainActivity : AppCompatActivity() {
50     override fun onCreate(savedInstanceState: Bundle?) {
51         super.onCreate(savedInstanceState)
52         setContent {
53             MaterialTheme {
54                 AppNavigation()
55             }
56         }
57     }
58 }
59
60 @Composable
61 fun AppNavigation() {
62     val navController = rememberNavController()
63
64     NavHost(navController = navController,
startDestination = "home") {
65         composable("home") { HomeScreen(navController) }
66         composable(
67             route = "detail/{songId}",
68             arguments = listOf(navArgument("songId") {
69                 type = NavType.IntType
70             })
71         ) { backStackEntry ->
72             val songId = backStackEntry.arguments?.getInt("songId")
73             DetailScreen(navController, songId)
74         }
75
76         composable("language") {
77             LanguageScreen(navController) }
78     }

```

```

79
80 @Composable
81 fun HomeScreen(
82     navController: NavController,
83     viewModel: SongViewModel = viewModel(factory =
SongViewModelFactory("Koleksi Music Compose"))
84 ) {
85     val context = LocalContext.current
86     val songList by viewModel.songList.collectAsState()
87     val navEvent by
viewModel.navigationEvent.collectAsState()
88     val intentEvent by
viewModel.intentEvent.collectAsState()
89
90     LaunchedEffect(navEvent) {
91         navEvent?.let { song ->
92             navController.navigate("detail/${song.id}")
93             viewModel.onNavigationHandled()
94         }
95     }
96
97     LaunchedEffect(intentEvent) {
98         intentEvent?.let { link ->
99             val intent = Intent(Intent.ACTION_VIEW,
Uri.parse(link))
100             context.startActivity(intent)
101             viewModel.onIntentHandled()
102         }
103     }
104
105     Column(
106         modifier = Modifier
107             .fillMaxSize()
108             .background(LavenderBlush)
109     ) {
110         Row(
111             modifier = Modifier
112                 .fillMaxWidth()
113                 .padding(16.dp),
114             horizontalArrangement =
Arrangement.SpaceBetween,
115             verticalAlignment =
Alignment.CenterVertically
116         ) {
117             Text(

```

```

118         text = stringResource(id =
R.string.app_name),
119         color = Color.Black,
120         fontSize = 22.sp,
121         fontWeight = FontWeight.Bold
122     )
123     IconButton(onClick = {
124         navController.navigate("language")
125     }) {
126         Icon(
127             painter =
painterResource(android.R.drawable.ic_menu_sort_alphabet
ically),
128             contentDescription = "Change
Language",
129             tint = Watermelon
130         )
131     }
132 }
133
134     LazyRow(contentPadding = PaddingValues(horizontal
= 16.dp),
135         horizontalArrangement =
Arrangement.spacedBy(16.dp)) {
136         items(songList) { song ->
137             SongItemCard(
138                 song = song,
139                 onDetailClick = {
viewModel.onDetailClicked(song) },
140                 onLinkClick = {
viewModel.onIntentClicked(song.externalLink) },
141                 Modifier.fillParentMaxWidth())
142             }
143         }
144
145         Spacer(modifier = Modifier.height(16.dp))
146
147         LazyColumn(contentPadding = PaddingValues(start =
16.dp, end = 16.dp, bottom = 16.dp),
148             verticalArrangement =
Arrangement.spacedBy(16.dp)) {
149             items(songList) { song ->
150                 SongItemCard(
151                     song = song,
152                     onDetailClick = {
viewModel.onDetailClicked(song) },

```

```

153         onLinkClick = {
154             viewModel.onIntentClicked(song.externalLink) },
155         Modifier.fillMaxWidth()
156     }
157 }
158 }
159
160 @Composable
161 fun SongItemCard(song: Song, onDetailClick: () -> Unit,
162     onLinkClick: () -> Unit, modifier: Modifier = Modifier) {
163     Card(
164         shape = RoundedCornerShape(16.dp),
165         colors = CardDefaults.cardColors(containerColor =
166             PinkChampagne),
167         elevation =
168             CardDefaults.cardElevation(defaultElevation = 4.dp),
169         modifier = modifier
170     ) {
171         Row(modifier =
172             Modifier.fillMaxWidth().padding(12.dp)) {
173             Image(
174                 painter = painterResource(id =
175                     song.imageResId),
176                 contentDescription = null,
177                 contentScale = ContentScale.Crop,
178                 modifier =
179                     Modifier.size(110.dp).clip(RoundedCornerShape(16.dp))
180             )
181             Spacer(modifier = Modifier.width(12.dp))
182
183             Column(modifier = Modifier.fillMaxWidth()) {
184                 Text(
185                     song.title,
186                     color = Color.Black,
187                     fontWeight = FontWeight.Bold,
188                     fontSize = 15.sp)
189                 Spacer(modifier = Modifier.height(4.dp))
190                 Text(text =
191                     "${stringResource(R.string.label_album)}
192                     ${song.albumName}", color = Color.Black, fontSize = 14.sp)
193
194                 Spacer(modifier = Modifier.height(8.dp))
195                 Row(horizontalArrangement =
196                     Arrangement.End, modifier = Modifier.fillMaxWidth()) {
197                     Button(

```

```

189         onClick = onLinkClick,
190         colors =
ButtonDefaults.buttonColors(containerColor = Color.White,
contentColor = Watermelon),
191         contentPadding =
PaddingValues(horizontal = 12.dp, vertical = 4.dp)
192     ) {
Text(stringResource(R.string.btn_listen),    fontSize =
12.sp) }
193
194         Spacer(modifier =
Modifier.width(8.dp))
195
196         Button(
197             onClick = onDetailClick,
198             colors =
ButtonDefaults.buttonColors(containerColor = Watermelon,
contentColor = Color.White),
199             contentPadding =
PaddingValues(horizontal = 12.dp, vertical = 4.dp)
200         ) {
Text(stringResource(R.string.btn_detail),    fontSize =
12.sp) }
201     }
202 }
203 }
204 }
205 }
206
207
208 @Composable
209 fun DetailScreen(navController: NavController, songId:
Int?) {
210     val song = SongData.songs.find { it.id == songId }
211
212     if (song != null) {
213         Column(
214             modifier = Modifier
215                 .fillMaxSize()
216                 .background(LavenderBlush)
217                 .verticalScroll(rememberScrollState())
218                 .padding(16.dp)
219         ) {
220             Box(
221                 modifier = Modifier
222                     .fillMaxWidth(),

```

```

223         contentAlignment = Alignment.Center
224     ) {
225         Image(
226             painter = painterResource(id =
song.imageResId),
227             contentDescription = null,
228             contentScale = ContentScale.Crop,
229             modifier = Modifier
230                 .size(320.dp)
231                 .clip(RoundedCornerShape(16.dp))
232         )
233     }
234
235     Spacer(modifier = Modifier.height(16.dp))
236     Text(
237         song.title,
238         color = Color.Black,
239         fontWeight = FontWeight.Bold,
240         fontSize = 28.sp
241     )
242     Text(
243         "${stringResource(R.string.label_album)}
${song.albumName}",
244         color = Color.Black,
245         fontSize = 16.sp,
246         modifier = Modifier.padding(top = 4.dp)
247     )
248     Text(
249         "${stringResource(R.string.label_year)}
${song.year}",
250         color = Color.Black,
251         fontSize = 16.sp,
252         modifier = Modifier.padding(top = 4.dp)
253     )
254     Text(
255         text
=
stringResource(song.descriptionResId),
256         color = Color.Black,
257         fontSize = 15.sp,
258         lineHeight = 22.sp,
259         modifier = Modifier.padding(top = 24.dp)
260     )
261
262     Spacer(modifier = Modifier.height(32.dp))
263     Button(

```

```

264         onClick = {
navController.popBackStack()},
265         colors =
ButtonDefaults.buttonColors(containerColor = Watermelon,
contentColor = Color.White),
266         modifier = Modifier.fillMaxWidth()
267     ) { Text(stringResource(R.string.btn_back)) }
268     }
269 }
270 }
271
272 @Composable
273 fun LanguageScreen(navController: NavController) {
274     Column(
275         modifier = Modifier
276             .fillMaxSize()
277             .background(LavenderBlush)
278             .padding(24.dp),
279         verticalArrangement = Arrangement.Center,
280         horizontalAlignment = Alignment.CenterHorizontally
281     ) {
282         Text(
283             text = stringResource(R.string.setting_language),
284             color = Color.Black,
285             fontSize = 24.sp,
286             fontWeight = FontWeight.Bold,
287             modifier = Modifier.padding(bottom = 24.dp)
288         )
289         Button(
290             onClick = { setAppLocale("en") },
291             colors = ButtonDefaults.buttonColors(containerColor = Watermelon,
contentColor = Color.White),
292             modifier = Modifier.fillMaxWidth()
293         ) { Text("English") }
294
295         Spacer(modifier = Modifier.height(16.dp))
296         Button(
297             onClick = { setAppLocale("id") },
298             colors = ButtonDefaults.buttonColors(containerColor = Watermelon,
contentColor = Color.White),
299             modifier = Modifier.fillMaxWidth()
300         ) { Text("Indonesia") }

```

301	}	
302	}	
303		
304	fun setAppLocale(languageTag: String) {	
305	val localeList	=
	LocaleListCompat.forLanguageTags(languageTag)	
306	AppCompatActivity.setApplicationLocales(localeList)	
307	}	

Tabel 4. Source Code Song.kt (XML)

1	package com.example.modul3_xml
2	
3	import java.io.Serializable
4	
5	data class Song(
6	val id: Int,
7	val title: String,
8	val albumName: String,
9	val year: String,
10	val imageResId: Int,
11	val externalLink: String,
12	val descriptionResId: Int
13	) : Serializable

Tabel 5. Source Code SongData.kt (XML)

1	package com.example.modul3_xml
2	
3	object SongData {
4	val songs = listOf(
5	Song(
6	1,
7	"Nothing's Gonna Hurt You Baby",
8	"I",
9	"2012",
10	R.drawable.album_i,
11	"https://open.spotify.com/track/3W7KHojYGgYaoX9ogKO9hU?si=bc234cdf897d49b3",
12	R.string.desc_i
13	),
14	Song(
15	2,
16	"Flash",
17	"Cigarettes After Sex",

```

18         "2017",
19         R.drawable.album_cas,
20
21         "https://open.spotify.com/track/6eknHC7HjAKchl23x2A7dW?s
22         i=7ed322498ba447be",
23         R.string.desc_flash
24     ),
25     Song(
26         3,
27         "Opera House",
28         "Cigarettes After Sex",
29         "2017",
30         R.drawable.album_cas,
31
32         "https://open.spotify.com/track/2Ddfm2NQwTRsf7YVlt258S?s
33         i=f70b90493a034663",
34         R.string.desc_opera
35     ),
36     Song(
37         4,
38         "Sweet",
39         "Cigarettes After Sex",
40         "2017",
41         R.drawable.album_cas,
42
43         "https://open.spotify.com/track/2KhrPRV0V1FS2l4eQMJUWt?s
44         i=77d6282cd2044d80",
45         R.string.desc_sweet
46     ),
47     Song(
48         5,
49         "Sesame Syrup",
50         "Crush",
51         "2018",
52         R.drawable.album_crush,
53
54         "https://open.spotify.com/track/6GcZ4pdtZgbf2ptgHG31bq?s
55         i=953557105c374872",
56         R.string.desc_sesame
57     ),
58     Song(
59         6,
60         "Neon Moon",
61         "Neon Moon",
62         "2018",
63         R.drawable.album_neon_moon,

```

56	"https://open.spotify.com/track/5r7QVaDpYeWMcbxxrkqb6H?si=489aa904e83f47f1",
57	R.string.desc_neon
58	),
59	Song(
60	7,
61	"Cry",
62	"Cry",
63	"2019",
64	R.drawable.album_cry,
65	
	"https://open.spotify.com/track/7mDTvYD2ieE4Q28XFziMfJ?si=89deal8975c047e9",
66	R.string.desc_cry
67	)
68	)
69	}

Tabel 6. Source Code strings.xml eng (XML)

1	<resources>
2	<string name="app_name">CAS Music</string>
3	<string name="label_album">Album:</string>
4	<string name="label_year">Release Year:</string>
5	<string name="btn_listen">Listen</string>
6	<string name="btn_detail">Detail</string>
7	<string name="btn_back">Back to Home</string>
8	<string name="setting_language">Select
9	Language</string>
10	<string name="desc_i">One of the most popular songs by CAS, this track acts as a \'grown-up\' lullaby and a promise of deep protection between two lovers. Recorded as an experimental improvisation in a university stairway, it tenderly captures the sweetness of their shared moments and a vow to safeguard each other from whatever the future holds.</string>
11	<string name="desc_flash">Following two lonely souls who meet on a rainy night, this song captures a fleeting yet powerful encounter. They find an instant connection and escape into a temporary bubble of happiness before reality inevitably pulls them back, leaving them with a beautiful memory of a glimpse of heaven amidst the chaos.</string>
12	<string name="desc_opera">Driven by deep passion and admiration, this song expresses a love so profound that the singer is willing to build an entire opera house in a

	secluded place just to properly convey the magnitude of his feelings to the one he loves.</string>
13	<string name="desc_sweet">This track beautifully describes two lovers who share a mutual connection of true love, highlighting the pure sweetness of knowing your partner loves you deeply without any need for second-guessing or overthinking.</string>
14	<string name="desc_sesame">Serving as the second single before their upcoming album, this ambient dream-pop track depicts sensual musings about a current lover, featuring the signature intimate and evocative lyrics typical of the band.</string>
15	<string name="desc_neon">A mesmerizing cover of the 1992 Brooks & Dunn country track, this song beautifully tells the heartbreaking story of a man who has lost the love of his life and is now stranded in a state of misery and pain, unable to move on.</string>
16	<string name="desc_cry">Told from the perspective of the less affectionate partner, this song delves into the dynamics of a toxic relationship where an imbalance in giving and receiving affection constantly leads to pain and heartache.</string>
17	</resources>

Tabel 7. Source Code strings.xml id (XML)

1	<resources>
2	<string name="app_name">Musik CAS</string>
3	<string name="label_album">Album:</string>
4	<string name="label_year">Tahun Rilis:</string>
5	<string name="btn_listen">Dengar</string>
6	<string name="btn_detail">Detail</string>
7	<string name="btn_back">Kembali ke Beranda</string>
8	<string name="setting_language">Pilih Bahasa</string>
9	
10	<string name="desc_i">Salah satu lagu paling populer dari CAS, lagu ini berfungsi sebagai pengantar tidur bagi orang dewasa dan janji perlindungan mendalam antara dua kekasih. Direkam sebagai improvisasi eksperimental di tangga universitas, lagu ini dengan lembut menangkap manisnya momen kebersamaan mereka dan sumpah untuk saling menjaga dari apa pun yang terjadi di masa depan.</string>
11	<string name="desc_flash">Mengisahkan dua jiwa kesepian yang bertemu di malam hujan, lagu ini menangkap pertemuan singkat namun penuh kekuatan. Mereka menemukan koneksi instan dan melarikan diri ke dalam gelembung kebahagiaan sementara sebelum kenyataan akhirnya menarik

	mereka kembali, meninggalkan kenangan indah tentang sekilas surga di tengah kekacauan.</string>
12	<string name="desc_opera">Didorong oleh gairah dan kekaguman yang mendalam, lagu ini mengekspresikan cinta yang begitu besar sehingga penyanyinya rela membangun seluruh gedung opera di tempat terpencil hanya untuk menyampaikan besarnya perasaannya kepada orang yang dicintainya.</string>
13	<string name="desc_sweet">Lagu ini dengan indah menggambarkan dua kekasih yang berbagi koneksi cinta sejati, menyoroti kemurnian manis saat mengetahui pasanganmu sangat mencintaimu tanpa perlu ragu atau terlalu banyak berpikir.</string>
14	<string name="desc_sesame">Sebagai singel kedua sebelum album mendatang mereka, lagu ambient dream-pop ini menggambarkan renungan sensual tentang kekasih saat ini, menampilkan lirik intim dan evokatif khas dari band tersebut.</string>
15	<string name="desc_neon">Sebuah cover memukau dari lagu country Brooks & Dunn tahun 1992, lagu ini dengan indah menceritakan kisah memilukan tentang seorang pria yang kehilangan cinta dalam hidupnya dan kini terdampar dalam penderitaan dan rasa sakit, tak mampu melangkah maju.</string>
16	<string name="desc_cry">Diceritakan dari sudut pandang pasangan yang kurang penyayang, lagu ini menggali dinamika hubungan toksik di mana ketidakseimbangan dalam memberi dan menerima kasih sayang terus-menerus berujung pada rasa sakit dan patah hati.</string>
17	</resources>

Tabel 8. Source Code item_song.xml (XML)

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_margin="8dp"
8	app:cardCornerRadius="16dp"
9	app:cardElevation="4dp"
10	app:cardBackgroundColor="#FFC2D1">
11	
12	<androidx.constraintlayout.widget.ConstraintLayout
13	android:layout_width="match_parent"

```

14         android:layout_height="wrap_content"
15         android:padding="12dp">
16
17     <com.google.android.material.imageview.ShapeableImageVie
18         w
19         android:id="@+id/img_song"
20         android:layout_width="110dp"
21         android:layout_height="110dp"
22         android:scaleType="centerCrop"
23
24     app:layout_constraintStart_toStartOf="parent"
25         app:layout_constraintTop_toTopOf="parent"
26
27     app:shapeAppearanceOverlay="@style/RoundedCornerImage"/>
28
29     <TextView
30         android:id="@+id/tv_title"
31         android:layout_width="0dp"
32         android:layout_height="wrap_content"
33         android:layout_marginStart="12dp"
34         android:textColor="@color/black"
35         android:textSize="16sp"
36         android:textStyle="bold"
37
38     app:layout_constraintEnd_toStartOf="@+id/tv_year"
39
40     app:layout_constraintStart_toEndOf="@+id/img_song"
41         app:layout_constraintTop_toTopOf="parent"/>
42
43     <TextView
44         android:id="@+id/tv_year"
45         android:layout_width="wrap_content"
46         android:layout_height="wrap_content"
47         android:textColor="@color/black"
48         app:layout_constraintEnd_toEndOf="parent"
49         app:layout_constraintTop_toTopOf="parent"/>
50
51     <TextView
52         android:id="@+id/tv_label_album"
53         android:layout_width="wrap_content"
54         android:layout_height="wrap_content"
55         android:layout_marginTop="8dp"
56         android:text="@string/label_album"
57         android:textColor="@color/black"
58
59     app:layout_constraintStart_toStartOf="@+id/tv_title"
60
61     app:layout_constraintTop_toBottomOf="@+id/tv_title"/>
62

```

55	<TextView
56	android:id="@+id/tv_album_name"
57	android:layout_width="0dp"
58	android:layout_height="wrap_content"
59	android:layout_marginStart="4dp"
60	android:textColor="@color/black"
61	app:layout_constraintEnd_toEndOf="parent"
62	
	app:layout_constraintStart_toEndOf="@+id/tv_label_album"
63	
	app:layout_constraintTop_toTopOf="@+id/tv_label_album"/>
64	
65	<Button
66	android:id="@+id/btn_link"
67	android:layout_width="wrap_content"
68	android:layout_height="wrap_content"
69	android:layout_marginEnd="8dp"
70	android:text="@string/btn_listen"
71	android:textColor="#FB6F92"
72	android:backgroundTint="@color/white"
73	
	app:layout_constraintBottom_toBottomOf="@+id/btn_detail"
74	
	app:layout_constraintEnd_toStartOf="@+id/btn_detail"
75	
	app:layout_constraintTop_toTopOf="@+id/btn_detail"/>
76	
77	<Button
78	android:id="@+id/btn_detail"
79	android:layout_width="wrap_content"
80	android:layout_height="wrap_content"
81	android:text="@string/btn_detail"
82	android:textColor="@color/white"
83	android:backgroundTint="#FB6F92"
84	
	app:layout_constraintBottom_toBottomOf="parent"
85	app:layout_constraintEnd_toEndOf="parent"
86	
	app:layout_constraintTop_toBottomOf="@+id/tv_album_name"
	/>
87	</androidx.constraintlayout.widget.ConstraintLayout>
88	
89	</com.google.android.material.card.MaterialCardView>

Tabel 9. Source Code fragment_home.xml (XML)

1	<?xml version="1.0" encoding="utf-8"?>
---	--

```

2 <LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     android:orientation="vertical"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     android:background="#FFE5EC">
8
9     <RelativeLayout
10         android:layout_width="match_parent"
11         android:layout_height="wrap_content"
12         android:padding="16dp">
13         <TextView
14             android:layout_width="wrap_content"
15             android:layout_height="wrap_content"
16             android:text="@string/app_name"
17             android:textSize="22sp"
18             android:textStyle="bold"/>
19
20         <ImageButton
21             android:id="@+id/btn_language"
22             android:layout_width="wrap_content"
23             android:layout_height="wrap_content"
24             android:layout_alignParentEnd="true"
25
26             android:background="?attr/selectableItemBackgroundBorderless"
27
28             android:src="@android:drawable/ic_menu_sort_alphabetically"
29
30             app:tint="#FB6F92"/>
31         </RelativeLayout>
32
33         <androidx.recyclerview.widget.RecyclerView
34             android:id="@+id/rv_highlight"
35             android:layout_width="match_parent"
36             android:layout_height="wrap_content"
37             android:orientation="horizontal"
38             android:paddingStart="8dp"
39             android:paddingEnd="8dp"
40             android:clipToPadding="false"/>
41
42         <androidx.recyclerview.widget.RecyclerView
43             android:id="@+id/rv_songs"
44             android:layout_width="match_parent"
45             android:layout_height="match_parent"
46             android:layout_marginTop="8dp"/>

```

45	</LinearLayout>
----	-----------------

Tabel 10. Source Code fragment_detail.xml (XML)

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.core.widget.NestedScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:background="#FFE5EC">
7	
8	<LinearLayout
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content"
11	android:orientation="vertical"
12	android:padding="16dp">
13	
14	<com.google.android.material.imageview.ShapeableImageVie
	w
15	android:id="@+id/img_detail_cover"
16	android:layout_width="320dp"
17	android:layout_height="320dp"
18	android:layout_gravity="center"
19	android:scaleType="centerCrop"
20	app:shapeAppearanceOverlay="@style/RoundedCornerImage"/>
21	
22	<TextView
23	android:id="@+id/tv_detail_title"
24	android:layout_width="match_parent"
25	android:layout_height="wrap_content"
26	android:layout_marginTop="16dp"
27	android:textSize="28sp"
28	android:textStyle="bold"/>
29	
30	<TextView
31	android:id="@+id/tv_detail_album"
32	android:layout_width="match_parent"
33	android:layout_height="wrap_content"
34	android:layout_marginTop="4dp"
35	android:textColor="@color/black"
36	android:textSize="16sp"/>
37	

38	<TextView
39	android:id="@+id/tv_detail_year"
40	android:layout_width="match_parent"
41	android:layout_height="wrap_content"
42	android:layout_marginTop="4dp"
43	android:textColor="@color/black"
44	android:textSize="16sp"/>
45	
46	<TextView
47	android:id="@+id/tv_detail_desc"
48	android:layout_width="match_parent"
49	android:layout_height="wrap_content"
50	android:layout_marginTop="24dp"
51	android:textColor="@color/black"
52	android:textSize="15sp"
53	android:lineSpacingExtra="6dp"/>
54	
55	<Button
56	android:id="@+id/btn_back"
57	android:layout_width="match_parent"
58	android:layout_height="wrap_content"
59	android:layout_marginTop="32dp"
60	android:backgroundTint="#FB6F92"
61	android:textColor="@color/white"
62	android:text="@string/btn_back"/>
63	</LinearLayout>
64	</androidx.core.widget.NestedScrollView>

Tabel 11. Source Code fragment_language.xml (XML)

1	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
	d"
2	android:layout_width="match_parent"
3	android:layout_height="match_parent"
4	android:orientation="vertical"
5	android:gravity="center"
6	android:padding="24dp"
7	android:background="#FFE5EC">
8	
9	<TextView
10	android:layout_width="wrap_content"
11	android:layout_height="wrap_content"
12	android:text="@string/setting_language"
13	android:textStyle="bold"
14	android:textSize="24sp"
15	android:layout_marginBottom="32dp"/>

```

16
17     <Button
18         android:id="@+id/btn_english"
19         android:layout_width="match_parent"
20         android:layout_height="wrap_content"
21         android:backgroundTint="#FB6F92"
22         android:textColor="@android:color/white"
23         android:text="English"/>
24
25     <Button
26         android:id="@+id/btn_indonesia"
27         android:layout_width="match_parent"
28         android:layout_height="wrap_content"
29         android:backgroundTint="#FB6F92"
30         android:textColor="@android:color/white"
31         android:text="Bahasa Indonesia"/>
32 </LinearLayout>

```

Tabel 12. Source Code SongAdapter.kt (XML)

```

1 package com.example.modul3_xml
2
3 import android.view.LayoutInflater
4 import android.view.ViewGroup
5 import androidx.recyclerview.widget.RecyclerView
6 import com.example.modul3_xml.databinding.ItemSongBinding
7
8 class SongAdapter(
9     private var list: List<Song>,
10    private val onDetailClick: (Song) -> Unit,
11    private val onLinkClick: (String) -> Unit
12 ) : RecyclerView.Adapter<SongAdapter.ViewHolder>() {
13
14     class ViewHolder(val binding: ItemSongBinding) :
15     RecyclerView.ViewHolder(binding.root)
16
17     override fun onCreateViewHolder(parent: ViewGroup,
18     viewType: Int): ViewHolder {
19         val binding =
20         ItemSongBinding.inflate(LayoutInflater.from(parent.context),
21         parent, false)
22         return ViewHolder(binding)
23     }
24
25     override fun onBindViewHolder(holder: ViewHolder,
26     position: Int) {
27         val song = list[position]
28     }
29 }

```

```

23         holder.binding.apply {
24             imgSong.setImageResource(song.imageResId)
25             tvTitle.text = song.title
26             tvAlbumName.text = song.albumName
27
28             btnDetail.setOnClickListener {
29                 onDetailClick(song)
30             }
31             btnLink.setOnClickListener {
32                 onLinkClick(song.externalLink)
33             }
34         }
35     }
36
37     override fun getItemCount(): Int {
38         return list.size
39     }
40
41     fun updateData(newList: List<Song>) {
42         list = newList
43         notifyDataSetChanged()
44     }
45 }

```

Tabel 13. Source Code HomeFragment.kt (XML)

```

1 package com.example.modul3_xml
2
3 import android.content.Intent
4 import android.net.Uri
5 import android.os.Bundle
6 import android.view.LayoutInflater
7 import android.view.View
8 import android.view.ViewGroup
9 import androidx.fragment.app.Fragment
10 import androidx.lifecycle.Lifecycle
11 import androidx.lifecycle.ViewModelProvider
12 import androidx.lifecycle.lifecycleScope
13 import androidx.lifecycle.repeatOnLifecycle
14 import androidx.navigation.fragment.findNavController
15 import androidx.recyclerview.widget.LinearLayoutManager
16 import androidx.recyclerview.widget.PagerSnapHelper
17 import
18 com.example.modul3_xml.databinding.FragmentHomeBinding
19 import kotlinx.coroutines.launch

```

```

20 class HomeFragment : Fragment() {
21     private var _binding: FragmentHomeBinding? = null
22     private val binding get() = _binding!!
23
24     private lateinit var viewModel: SongViewModel
25     private lateinit var adapter: SongAdapter
26
27     override fun onCreateView(
28         inflater: LayoutInflater,
29         container: ViewGroup?,
30         savedInstanceState: Bundle?
31     ): View {
32         _binding = FragmentHomeBinding.inflate(inflater,
container, false)
33         return binding.root
34     }
35
36     override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
37         super.onViewCreated(view, savedInstanceState)
38
39         val factory = SongViewModelFactory("Data Master
Music CAS")
40         viewModel = ViewModelProvider(this,
factory)[SongViewModel::class.java]
41
42         adapter = SongAdapter(emptyList(),
43             onDetailClick = { song ->
viewModel.onDetailClicked(song) },
44             onLinkClick = { url ->
viewModel.onIntentClicked(url) }
45         )
46
47         binding.rvSongs.layoutManager =
LinearLayoutManager(requireContext())
48         binding.rvSongs.adapter = adapter
49
50         binding.rvHighlight.layoutManager =
LinearLayoutManager(requireContext(),
LinearLayoutManager.HORIZONTAL, false)
51         binding.rvHighlight.adapter = adapter
52         val snapHelper = PagerSnapHelper()
53         snapHelper.attachToRecyclerView(binding.rvHighlight)
54
55         binding.btnLanguage.setOnClickListener {

```

```

56     findNavController().navigate(R.id.action_home_to_language)
57     }
58
59     viewLifecycleOwner.lifecycleScope.launch {
60
61         viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
62             launch {
63                 viewModel.songList.collect { songs ->
64                     adapter.updateData(songs)
65                 }
66             }
67             launch {
68                 viewModel.navigationEvent.collect { song
69 ->
70                     song?.let {
71                         val bundle = Bundle().apply {
72                             putSerializable("song", it) }
73                         findNavController().navigate(R.id.action_home_to_detail,
74                             bundle)
75                         viewModel.onNavigationHandled()
76                     }
77                 }
78             }
79             launch {
80                 viewModel.intentEvent.collect { url ->
81                     url?.let {
82                         val intent =
83                         Intent(Intent.ACTION_VIEW, Uri.parse(it))
84                         startActivity(intent)
85                         viewModel.onIntentHandled()
86                     }
87                 }
88             }
89
90             override fun onDestroyView() {
91                 super.onDestroyView()
92                 _binding = null
93             }

```

Tabel 14. Source Code DetailFragment.kt (XML)

```

1 package com.example.modul3_xml
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import androidx.navigation.fragment.findNavController
9 import
com.example.modul3_xml.databinding.FragmentDetailBinding
10 class DetailFragment : Fragment() {
11     private var _binding: FragmentDetailBinding? = null
12     private val binding get() = _binding!!
13
14     override fun onCreateView(
15         inflater: LayoutInflater,
16         container: ViewGroup?,
17         savedInstanceState: Bundle?
18     ): View? {
19         _binding =
FragmentDetailBinding.inflate(inflater, container, false)
20         return binding.root
21     }
22
23     override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
24         super.onViewCreated(view, savedInstanceState)
25
26         val song = arguments?.getSerializable("song") as?
Song
27         if (song != null) {
28             binding.imgDetailCover.setImageResource(song.imageResId)
29             binding.tvDetailTitle.text = song.title
30
31             val labelAlbum =
getString(R.string.label_album)
32             val labelYear =
getString(R.string.label_year)
33
34             binding.tvDetailAlbum.text = "$labelAlbum
${song.albumName}"

```

35	binding.tvDetailYear.text = "\$labelYear \${song.year}"
36	binding.tvDetailDesc.text = getString(song.descriptionResId)
37	}
38	
39	binding.btnBack.setOnClickListener {
40	findNavController().navigateUp()
41	}
42	}
43	
44	override fun onDestroyView() {
45	super.onDestroyView()
46	_binding = null
47	}
48	}

Tabel 15. Source Code LanguageFragment.kt (XML)

1	package com.example.modul3_xml
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.appcompat.app.AppCompatActivityDelegate
8	import androidx.core.os.LocaleListCompat
9	import androidx.fragment.app.Fragment
10	import com.example.modul3_xml.databinding.FragmentLanguageBindi ng
11	class LanguageFragment : Fragment() {
12	private var _binding: FragmentLanguageBinding? = null
13	private val binding get() = _binding!!
14	
15	override fun onCreateView(16 inflater: LayoutInflater, 17 container: ViewGroup?, 18 savedInstanceState: Bundle? 19): View? {
20	_binding = FragmentLanguageBinding.inflate(inflater, container, false)
21	return binding.root
22	}
23	

24	override fun onViewCreated(view: View,
25	savedInstanceState: Bundle?) {
26	super.onViewCreated(view, savedInstanceState)
27	binding.btnEnglish.setOnClickListener {
28	setAppLocale("en")
29	}
30	binding.btnIndonesia.setOnClickListener {
31	setAppLocale("id")
32	}
33	}
34	
35	private fun setAppLocale(languageTag: String) {
36	val localeList =
37	LocaleListCompat.forLanguageTags(languageTag)
38	AppCompatActivity.setApplicationLocales(localeList)
39	}
40	override fun onDestroyView() {
41	super.onDestroyView()
42	_binding = null
43	}
44	}

Tabel 16. Source Code MainActivity.kt (XML)

1	package com.example.modul3_xml
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import
6	com.example.modul3_xml.databinding.ActivityMainBinding
7	class MainActivity : AppCompatActivity() {
8	private lateinit var binding: ActivityMainBinding
9	
10	override fun onCreate(savedInstanceState: Bundle?) {
11	super.onCreate(savedInstanceState)
12	binding =
13	ActivityMainBinding.inflate(layoutInflater)
14	setContentView(binding.root)
15	}

Tabel 17. Source Code Tambahan MusicApplication.kt (Compose)

1	package com.example.modul3_compose_baru
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	class MusicApplication : Application() {
7	override fun onCreate() {
8	super.onCreate()
9	Timber.plant(Timber.DebugTree())
10	}
11	}

Tabel 18. Source Code Tambahan SongViewModel.kt (Compose)

1	package com.example.modul3_compose_baru
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import kotlinx.coroutines.flow.MutableStateFlow
6	import kotlinx.coroutines.flow.StateFlow
7	import kotlinx.coroutines.flow.asStateFlow
8	import timber.log.Timber
9	
10	class SongViewModel (private val categoryName: String) :
	ViewModel() {
11	private val _songList =
	MutableStateFlow<List<Song>>(emptyList())
12	val songList: StateFlow<List<Song>> =
	_songList.asStateFlow()
13	
14	private val _navigationEvent =
	MutableStateFlow<Song?>(null)
15	val navigationEvent: StateFlow<Song?> =
	_navigationEvent.asStateFlow()
16	
17	private val _intentEvent =
	MutableStateFlow<String?>(null)
18	val intentEvent: StateFlow<String?> =
	_intentEvent.asStateFlow()
19	
20	init {
21	loadSongs()
22	}
23	

```

24     private fun loadSongs() {
25         Timber.d("[${categoryName}] Memuat data lagu ke
dalam UI Compose...")
26         _songList.value = SongData.songs
27         Timber.d("Berhasil memuat ${_songList.value.size}
item lagu.")
28     }
29
30     fun onDetailClicked(song: Song) {
31         Timber.d("Tombol Detail ditekan. Navigasi ke ID
Lagu: ${song.id} Judul: ${song.title}.")
32         _navigationEvent.value = song
33     }
34
35     fun onIntentClicked(link: String) {
36         Timber.d("Tombol Listen ditekan. Membuka link:
$link")
37         _intentEvent.value = link
38     }
39
40     fun onNavigationHandled() { _navigationEvent.value =
null }
41     fun onIntentHandled() { _intentEvent.value = null }
42 }
43
44 class SongViewModelFactory(private val categoryName:
String) : ViewModelProvider.Factory {
45     override fun <T : ViewModel> create(modelClass:
Class<T>): T {
46         if
47         (modelClass.isAssignableFrom(SongViewModel::class.java))
48         {
49             @Suppress("UNCHECKED_CAST")
50             return SongViewModel(categoryName) as T
51         }
52         throw IllegalArgumentException("Unknown ViewModel
class")
53     }
54 }

```

Tabel 19. Source Code Tambahan MusicApplication.kt (XML)

```

1 package com.example.modul3_xml
2
3 import android.app.Application

```

4	import timber.log.Timber
5	
6	class MusicApplication : Application() {
7	override fun onCreate() {
8	super.onCreate()
9	Timber.plant(Timber.DebugTree())
10	}
11	}

Tabel 20. Source Code Tambahan SongViewModel.kt (XML)

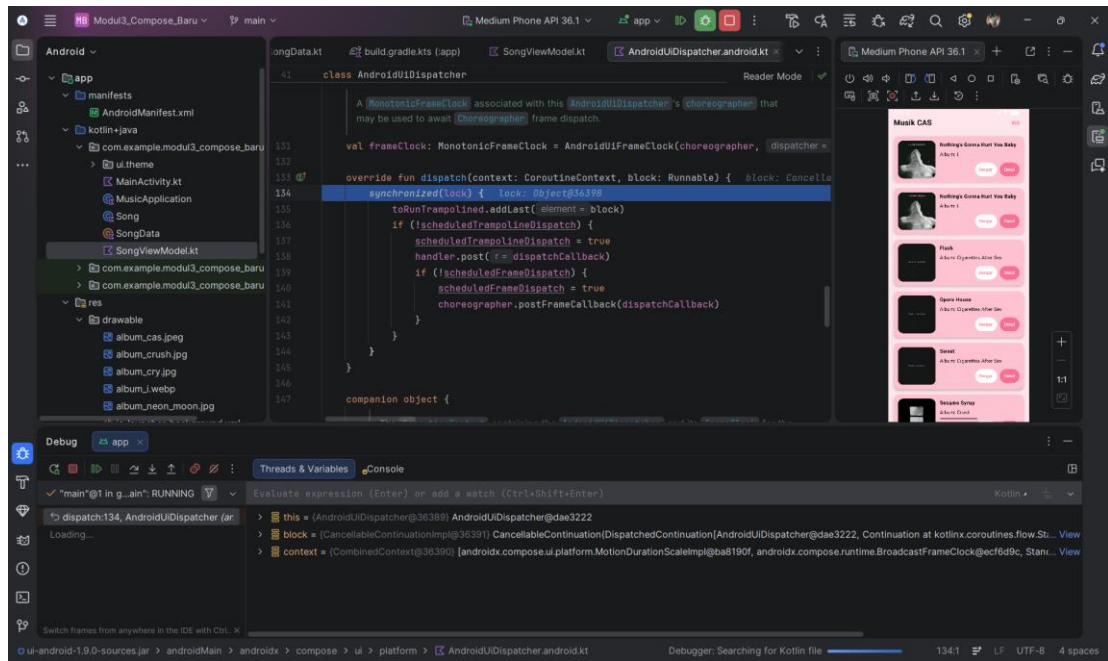
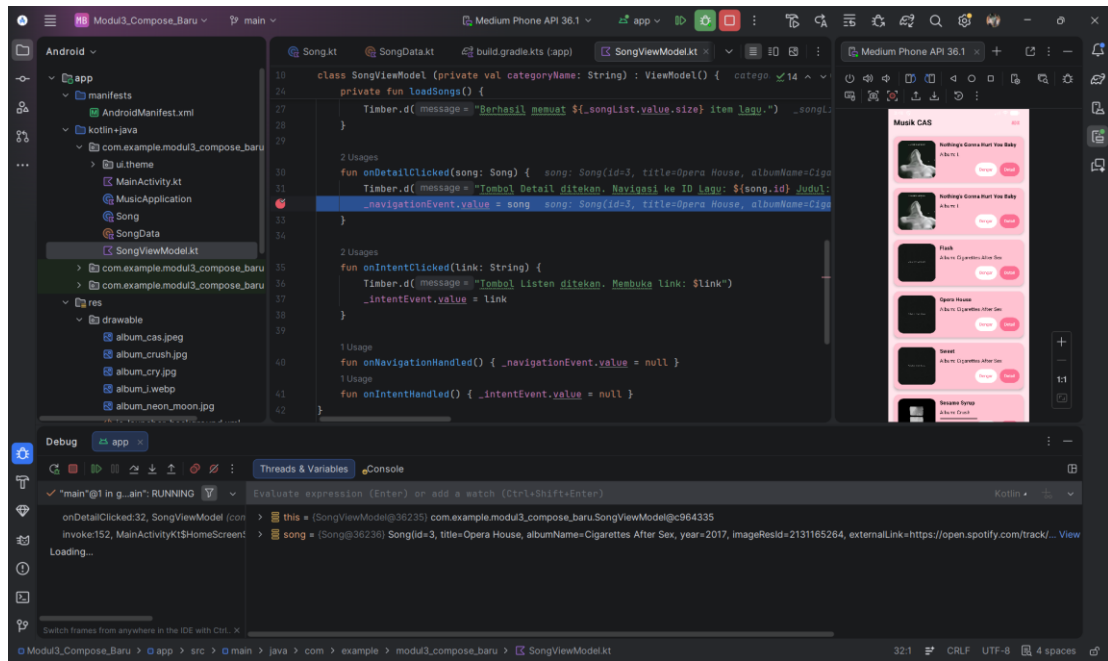
1	package com.example.modul3_xml
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import kotlinx.coroutines.flow.MutableStateFlow
6	import kotlinx.coroutines.flow.StateFlow
7	import kotlinx.coroutines.flow.asStateFlow
8	import timber.log.Timber
9	
10	class SongViewModel(private val categoryName: String) :
	ViewModel() {
11	private val _songList =
	MutableStateFlow<List<Song>>(emptyList())
12	val songList: StateFlow<List<Song>> =
	_songList.asStateFlow()
13	
14	private val _navigationEvent =
	MutableStateFlow<Song?>(null)
15	val navigationEvent: StateFlow<Song?> =
	_navigationEvent.asStateFlow()
16	
17	private val _intentEvent =
	MutableStateFlow<String?>(null)
18	val intentEvent: StateFlow<String?> =
	_intentEvent.asStateFlow()
19	
20	init {
21	loadSongs()
22	}
23	
24	private fun loadSongs() {
25	Timber.d("[\${categoryName}] Proses memasukkan data
	lagu ke dalam list dimulai...")
26	_songList.value = SongData.songs

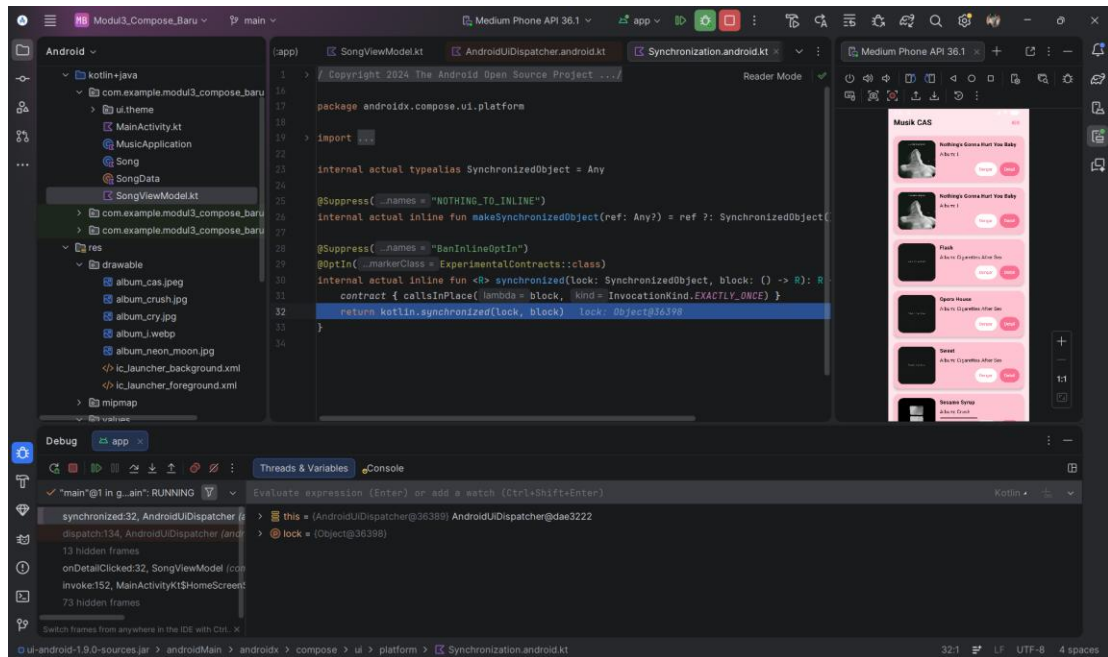
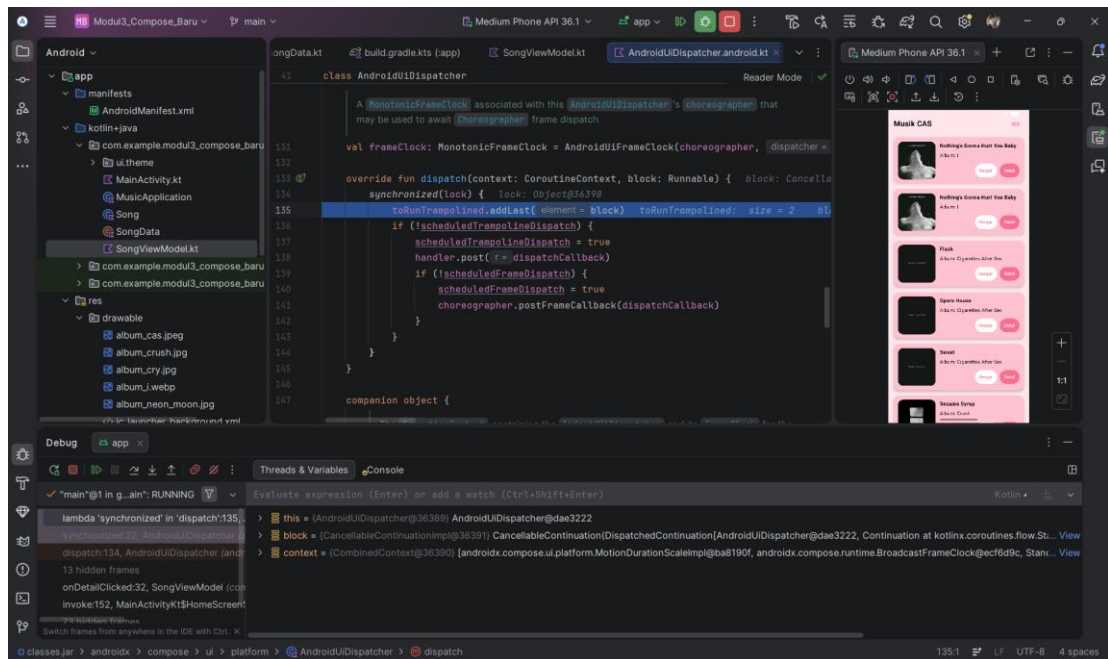
```

27         Timber.d("Berhasil memuat ${_songList.value.size}
data item lagu.")
28     }
29
30     fun onDetailClicked(song: Song) {
31         Timber.d("Tombol 'Detail' pada UI telah ditekan
pengguna.")
32         Timber.d("Mempersiapkan data Navigasi -> ID Lagu:
${song.id}, Judul: ${song.title}")
33         _navigationEvent.value = song
34     }
35
36     fun onIntentClicked(link: String) {
37         Timber.d("Tombol 'Listen' ditekan. Mengeksekusi
explicit intent ke external url: $link")
38
39         _intentEvent.value = link
40     }
41
42     fun onNavigationHandled() { _navigationEvent.value =
null }
43     fun onIntentHandled() { _intentEvent.value = null }
44 }
45
46 class SongViewModelFactory(private val categoryParam:
String) : ViewModelProvider.Factory {
47     override fun <T : ViewModel> create(modelClass:
Class<T>): T {
48         if
(modelClass.isAssignableFrom(SongViewModel::class.java))
49         {
50             @Suppress("UNCHECKED_CAST")
51             return SongViewModel(categoryParam) as T
52         }
53         throw IllegalArgumentException("Unknown ViewModel
class")
54     }
55 }

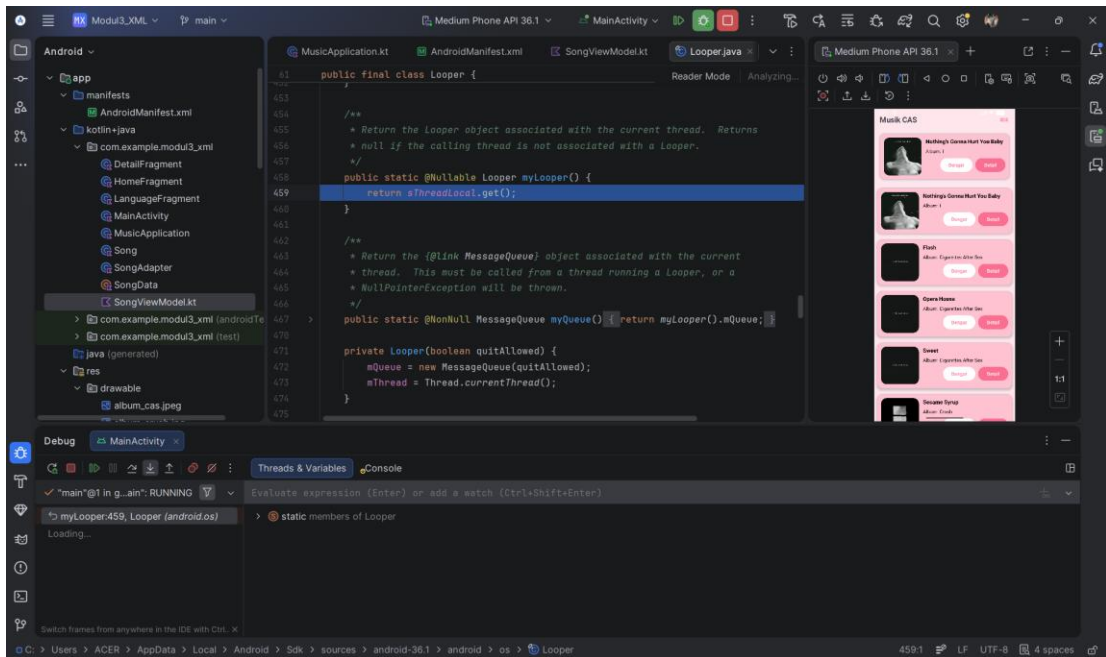
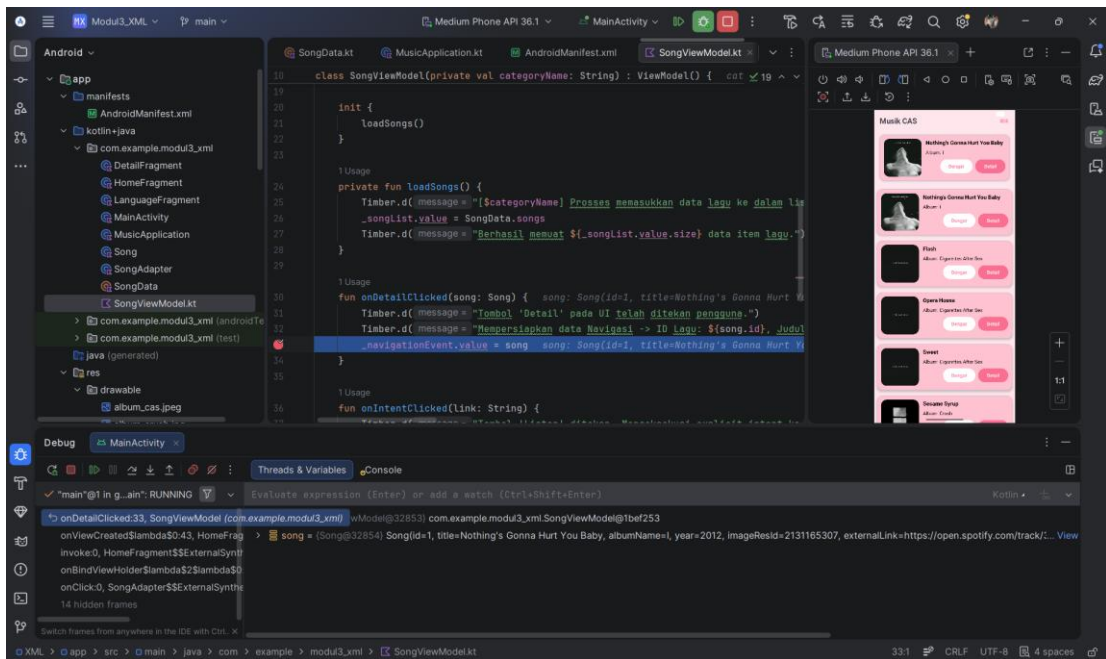
```

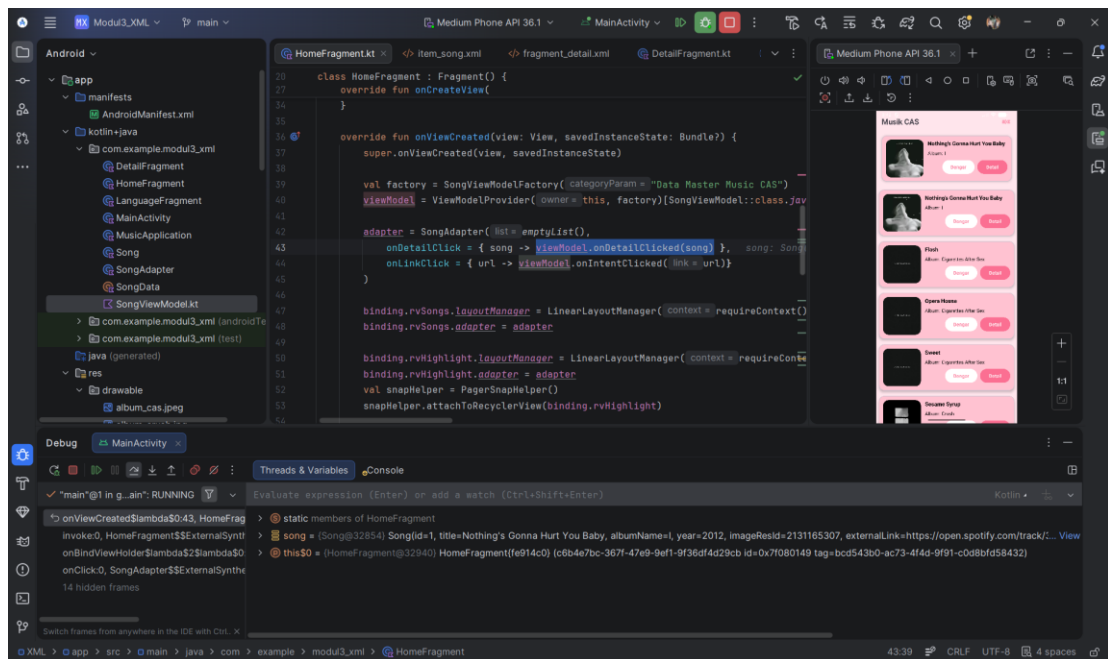
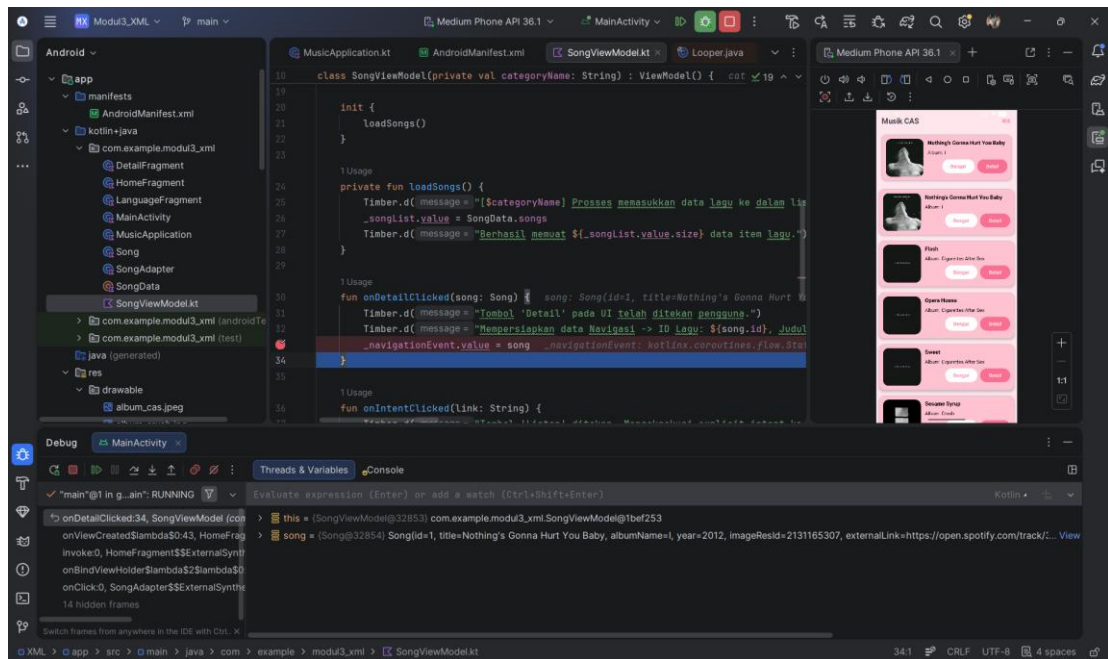
C. Output Program





Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose





Gambar 2. Screenshot Hasil Jawaban Soal 1 XML

D. Pembahasan

Song.Kt (Compose)

Pada baris [1] dan [3], deklarasi package menunjukkan lokasi direktori file, diikuti dengan pemanggilan pustaka bawaan Java yaitu `java.io.Serializable`.

Pada baris [5], dideklarasikan data class Song. Penggunaan kata kunci data menginstruksikan kompilator Kotlin untuk secara otomatis membuat kode berulang (boilerplate) tingkat utilitas seperti equals(), hashCode(), toString(), dan copy(). Hal ini menjadikan kelas ini murni berfokus sebagai wadah pembawa data.

Pada baris [6] sampai [12], properti title, albumName, year, dan externalLink menggunakan tipe data String. Sementara itu, id, imageResId, dan descriptionResId menggunakan tipe data Int. Penggunaan Int penting karena menyimpan ID referensi yang merujuk pada berkas fisik di dalam direktori res/drawable dan lokasi terjemahan di res/values/strings.xml

Pada baris [13], terdapat implementasi antarmuka : Serializable. Modifikasi ini memberikan kemampuan pada objek Song untuk dikonversi menjadi byte stream. Secara arsitektural, ini memungkinkan objek lagu untuk dikirim melintasi batas memori antar-komponen aplikasi, misalnya diteruskan melalui argumen sistem navigasi atau intent.

SongData.Kt(Compose)

Pada baris [3], object SongData. Berbeda dengan class biasa yang bisa diinstansiasi berulang kali, kata kunci object di Kotlin menerapkan design pattern Singleton. Ini menjamin bahwa di seluruh lifecycle aplikasi, memori perangkat hanya akan menyimpan satu buah salinan objek SongData ini saja. Pendekatan ini sangat efisien untuk menyediakan sumber data (*data source*) statis secara global.

Pada baris [4] sampai [69], variabel global val songs dideklarasikan yang diinisialisasi menggunakan fungsi listOf(...). Karena menggunakan listOf daripada mutableListOf, koleksi data yang dihasilkan bersifat *immutable* (tidak dapat dimodifikasi, ditambah, atau dihapus setelah aplikasi berjalan), memastikan data tetap aman dan konsisten saat dirender oleh LazyColumn atau LazyRow di lapisan UI. Di dalam blok listOf, dilakukan pemanggilan konstruktor Song(...) secara berulang untuk melakukan pemetaan *dummy data*. Setiap blok mengisi parameter sesuai kontrak data kelasnya secara berurutan, mengorganisasikan koleksi lagu.

MainActivity.kt (Compose)

Pada baris [1] sampai [43], package digunakan untuk mendeklarasikan direktori file program, sedangkan import digunakan untuk memanggil library dasar Android dan komponen Jetpack Compose. Pustaka yang dipanggil mencakup elemen UI dasar (Image, Text, Button), tata letak (Column, Row, LazyColumn, LazyRow), system navigasi (NavController, NavHost), serta library untuk fungsionalitas spesifik seperti Intent untuk membuka tautan eksternal dan AppCompatActivity untuk manajemen bahasa aplikasi.

Pada baris [45] sampai [47], dilakukan inisialisasi variabel global berupa val LavenderBlush, PinkChampagne, dan Watermelon menggunakan class Color. ini bertujuan untuk membuat palet warna yang diinginkan, lalu supaya warna antarmuka tetap konsisten dan memudahkan modifikasi tema di masa mendatang tanpa perlu mengubah kode hex satu per satu di setiap komponen.

Pada baris [49] sampai [58], class 'MainActivity : AppCompatActivity()' adalah deklarasi kelas utama. Terdapat perbedaan arsitektur, kelas ini mewarisi AppCompatActivity daripada ComponentActivity standar Compose. Hal ini diwajibkan karena aplikasi memanfaatkan fitur pergantian bahasa secara dinamis menggunakan AppCompatActivity, yang hanya didukung oleh arsitektur backward-compatible dari kelas tersebut. Di dalam metode lifecycle override fun onCreate(...), blok setContent {...} dipanggil untuk menggambar akar dari komponen UI, yaitu App Navigation.

Pada baris [60] sampai [78], @Composable fun AppNavigation berfungsi sebagai navigasi aplikasi. Dalam arsitektur Jetpack Compose, perpindahan antarlayar tidak lagi menggunakan Activity, melainkan menukar komponen Composable di dalam satu Activity Tunggal.

Pada baris [62], variabel navController diinisialisasi menggunakan rememberNavController(). Objek ini bertugas melacak posisi layar pengguna saat ini dan menyimpan riwayat tumpukan navigasi (back stack) agar pengguna bisa Kembali

ke layar sebelumnya. remember memastikan status navigasi ini tidak hilang saat terjadi recomposition (render ulang UI).

Pada baris [64], komponen NavHost membungkus seluruh rute layar aplikasi. Atribut `startDestination = "home"` secara eksplisit memerintahkan aplikasi untuk menampilkan layar beranda pertama kali saat dibuka.

Pada baris [65], fungsi `composable("home")` mendaftarkan rute statis yang akan memanggil fungsi `HomeScreen(navController)`.

Pada baris [66] sampai [74], ada implementasi dynamic routing. Rute didaftarkan sebagai `"detail/{songId}"`, yang berarti rute perlu sebuah parameter atau argument yang disisipkan pada URL internalnya. Atribut `arguments = listOf(...)` secara spesifik mendefinisikan bahwa argument `"songId"` yang ditangkap harus dikonversi dan diperlukan sebagai data Integer (`NavType.IntType`). Variable `backStackEntry` kemudian mengekstrak argument `songId` tersebut, lalu meneruskannya ke dalam `DetailScreen(navController, songId)`. Mekanisme ini memastikan layar detail merender lagu yang tepat sesuai dengan data yang ditekan pengguna di beranda.

Pada baris [76], rute statis `"language"` didaftarkan untuk mengakses menu pengaturan Bahasa.

Pada baris [80] sampai [158], `@Composable fun HomeScreen()` bertanggung jawab menyusun tata letak halaman utama aplikasi.

Pada baris [81] sampai [84], metode `composable HomeScreen` dimodifikasi dengan penambahan parameter `viewModel: SongViewModel = viewModel(factory = SongViewModelFactory(...))`. Ini adalah implementasi injeksi ViewModel secara langsung ke dalam layar menggunakan fungsi utilitas Jetpack Compose, di mana Factory yang dibuat sebelumnya dipanggil.

Pada baris [85], `val context = LocalContext.current` digunakan untuk mendapatkan context dari system operasi Android. Objek context ini wajib diteruskan ke

SongItemCard karena aksi membuka link (implicit intent) membutuhkan context untuk menjalankan komponen di luar aplikasi.

Pada baris [86] sampai [88], properti aliran data diamati di dalam lingkup UI menggunakan ekstensi `.collectAsState()`. Ini membuat Jetpack Compose selalu mendengarkan emisi data dari ViewModel dan melakukan recomposition (render ulang layar) jika datanya berubah.

Pada baris [90] sampai [103], blok `LaunchedEffect` diterapkan untuk mengamati status `navEvent` dan `intentEvent`. `LaunchedEffect` menjamin bahwa aksi side-effect yang berat seperti perpindahan layar menggunakan `navController.navigate` dan eksekusi `intent.startActivity(intent)` hanya dipicu satu kali per perubahan data dan berjalan aman di luar siklus perenderan visual Compose.

Pada baris [105] sampai [109], wadah utama aplikasi dibungkus menggunakan `Column` dengan modifikasi `fillMaxSize` agar mengisi layar penuh, serta diberi warna latar belakang `LavenderBlush`.

Pada baris [110] sampai [132], ini adalah header. Ada `Row` dengan modifikasi `fillMaxWidth()` dan `padding(16.dp)`. Atribut `Arrangement.SpaceBetween` sangat penting karena akan menolak elemen ke ujung kiri (teks judul) dan ujung kanan (tombol pengaturan). Komponen `Text` menampilkan nama aplikasi menggunakan `stringResource`, yang artinya teks ini mendukung multi language.

Pada baris [134] sampai [143], komponen `LazyRow` bertugas merender daftar lagu yang dapat digulir ke kiri dan kanan. `PaddingValues(horizontal = 16.dp)` memberikan jarak awal dan akhir agar elemen tidak menempel pada pinggir layar, `Arrangement.spacedBy(16.dp)` memberi spasi konsisten antarkartu. Lalu ada pemanggilan `SongItemCard` yang diberikan parameter `Modifier.fillParentMaxWidth()`. Modifikasi ini memaksa setiap kartu dalam daftar memiliki lebar sama persis dengan lebar layar perangkat, mengikuti dimensi parent vertical, sehingga ukurannya sama seperti kartu yang ada di daftar gulir atas-bawah.

Pada baris [139], [140], [152], dan [153], pemanggilan komponen `SongItemCard` pada `LazyRow` dan `LazyColumn` dimodifikasi. Logika navigasi dan intent yang tadinya dikelola langsung, kini dilempar ke atas dan diserahkan penanganannya kepada `ViewModel` melalui pemanggilan `viewModel.onDetailClicked(song)` dan `viewModel.onIntentClicked(song.externalLink)`.

Pada baris [147] sampai [156], komponen `LazyColumn` merender daftar lagu yang dapat digulir ke atas dan bawah. Dimodifikasi menggunakan `Modifier.fillMaxWidth()` secara standar, dan `items(SongData.songs)` melakukan perulangan dinamis untuk mencetak `SongItemCard` sebanyak jumlah lagu yang ada.

Pada baris [160] sampai [205], `@Composable fun SongItemCard()` adalah komponen antarmuka reusable untuk mendesain satu buah kartu lagu.

Pada baris [161], deklarasi `SongItemCard` dimodifikasi. Konstruktor antarmuka ini dipangkas sehingga menjadi komponen murni (stateless) yang menerima aksi interaksi berupa parameter fungsi lambda `onDetailClick: () -> Unit` dan `onLinkClick: () -> Unit`.

Pada baris [162] sampai [167], fondasi menggunakan `Card`. Kartu ini diberi `RoundedCornerShape(16.dp)` agar keempat sudutnya melengkung, dengan warna khusus `PinkChampagne`, juga diberi efek bayangan menggunakan `elevation` sebesar `4.dp`.

Pada baris [168], tata letak di dalam kartu dibelah menggunakan `Row`.

Pada baris [169] sampai [174], kartu diisi dengan komponen `Image` di sebelah kiri. Supaya gambar album proporsional dan tidak gepeng, digunakan property `contentScale = ContentScale.Crop`. Modifikasi `clip(RoundedCornerShape(16.dp))` memastikan gambar terpotong melengkung mengikuti bentuk kartu di luarnya.

Pada baris [177] sampai [184], sebelah kanan kartu diisi dengan `Column` yang memuat teks judul dan album. `Spacer` digunakan untuk merenggankan jarak vertikal antar-teks.

Pada baris [187] sampai [201], bagian ini menyusun tombol `Listen` dan `Detail` dalam satu baris di ujung kanan kartu menggunakan `Arrangement.End`. Tombol `Listen`

mengimplementasikan konsep implicit intent. Variabel intent menyimpan perintah `Intent.ACTION_VIEW` yang disertai konversi URL link lagu (`Uri.parse(song.externalLink)`). Saat tombol ditekan, `context.startActivity(intent)` akan diluncurkan, memaksa sistem operasi Android mencari aplikasi eksternal untuk membuka link tersebut. Kalau tombol Detail, tombol ini memicu navigasi internal aplikasi dengan mengeksekusi `navController.navigate("detail/${song.id}")`. Perintah ini menyisipkan ID spesifik lagu yang sedang ditekan ke dalam rute, nanti akan ditangkap oleh `AppNavigation()`.

Pada baris [208] sampai [270], `@Composable fun DetailScreen()` merender halaman informasi lengkap dari sebuah lagu yang dipilih.

Pada baris [180] dan [182], saat layar dimuat maka program langsung mencari data dari koleksi `SongData.song` menggunakan perintah `.find { it.id == songId }`. Ini memastikan antarmuka hanya akan menampilkan satu data objek tunggal yang ID-nya cocok dengan argumen yang diterima dari navigasi. Jika data ditemukan (`if (song != null)`), layar akan di-render.

Pada baris [213] sampai [219], konten utama dibungkus `Column`. Modifikasi `verticalScroll(rememberScrollState())` diimplementasikan pada kolom ini, agar keseluruhan deskripsi layar yang panjang dapat digulir ke bawah dan tidak terpotong di ujung layar perangkat.

Pada baris [220] sampai [233], komponen `Box` bertindak sebagai wadah pembungkus dengan `contentAlignment = Alignment.Center` agar gambar yang ada di dalamnya presisi berada di tengah layar secara horizontal. Ini menampilkan cover album.

Pada baris [236] sampai [260], komponen `Text` disusun vertikal untuk menampilkan judul, nama, album, tahun rilis, dan deskripsi lengkap. Properti `stringResource` dikombinasikan dengan iterasi data Kotlin seperti `"${stringResource(R.string.label_album)} ${song.albumName}"` agar label bahasa mendukung pergantian bahasa, sementara isi datanya dinamis mengikuti lagu yang dipilih.

Pada baris [263] sampai [267], ujung bawah halaman ditutup dengan Button aksi kembali. Pada atribut `onClick`, perintah `navController.popBackStack()` dipanggil. Perintah ini berfungsi secara literal untuk keluar dari layar detail, keluar dari tumpukan memori navigasi, sehingga pengguna otomatis dikembalikan ke tumpukan layar sebelumnya.

Pada baris [272] sampai [302], `@Composable` fun `LanguageScreen()` merender antarmuka khusus untuk panel pengaturan lokalisasi Bahasa.

Pada baris [274] sampai [281], keseluruhan layar dikendalikan oleh satu `Column` dengan parameter pengaturan `Arrangement.Center` (berpusat di tengah vertical) dan `Alignment.CenterHorizontally` (berpusat di tengah horizontal). Kombinasi ini memastikan seluruh isi konten terkunci secara simetris di tengah-tengah layar.

Pada baris [289] sampai [300], terdapat dua tombol, yaitu tombol `English` dan `Indonesia`. Tombol-tombol ini tidak melakukan perpindahan layar secara navigasi, melainkan fungsi khusus `setAppLocale("en")` dan `setAppLocale("id")` sebagai event trigger pergantian lokalisasi aplikasi.

Pada baris [304] sampai [307], fun `setAppLocale()` adalah fungsi utilitas procedural Kotlin yang mengurus back-end pelokalan system Android modern.

Pada baris [304], perintah `LocaleListCompat.forLanguageTags(languageTag)` menerima string kode bahasa (seperti `"en"` atau `"id"`) dan mengonversinya ke dalam bentuk daftar local (locale).

Pada baris [306], perintah `AppCompatActivity.setApplicationLocales(localeList)` kemudian menerapkan konfigurasi baru pada `Application Context` (tingkat aplikasi). Intruksi ini memaksa system untuk secara langsung mengambil nilai string dari file resource (`strings.xml`) yang berbeda dan memicu `Compose` untuk me-render ulang (`recomposition`) seluruh layar saat itu juga dengan bahasa yang baru, tanpa mengharuskan pengguna mematikan dan restart aplikasi.

Song.kt (XML)

Pada baris [1] dan [3], deklarasi package menunjukkan lokasi direktori file, diikuti dengan pemanggilan pustaka bawaan Java yaitu `java.io.Serializable`.

Pada baris [5], dideklarasikan data class `Song`. Penggunaan kata kunci data menginstruksikan kompilator Kotlin untuk secara otomatis membuat kode berulang (boilerplate) tingkat utilitas seperti `equals()`, `hashCode()`, `toString()`, dan `copy()`. Hal ini menjadikan kelas ini murni berfokus sebagai wadah pembawa data.

Pada baris [6] sampai [12], properti `title`, `albumName`, `year`, dan `externalLink` menggunakan tipe data `String`. Sementara itu, `id`, `imageResId`, dan `descriptionResId` menggunakan tipe data `Int`. Penggunaan `Int` penting karena menyimpan ID referensi yang merujuk pada berkas fisik di dalam direktori `res/drawable` dan lokasi terjemahan di `res/values/strings.xml`

Pada baris [13], terdapat implementasi antarmuka : `Serializable`. Modifikasi ini memberikan kemampuan pada objek `Song` untuk dikonversi menjadi byte stream. Secara arsitektural, ini memungkinkan objek lagu untuk dikirim melintasi batas memori antar-komponen aplikasi, misalnya diteruskan melalui argumen sistem navigasi atau intent.

SongData.kt (XML)

Pada baris [3], object `SongData`. Berbeda dengan class biasa yang bisa diinstansiasi berulang kali, kata kunci object di Kotlin menerapkan design pattern Singleton. Ini menjamin bahwa di seluruh lifecycle aplikasi, memori perangkat hanya akan menyimpan satu buah salinan objek `SongData` ini saja. Pendekatan ini sangat efisien untuk menyediakan sumber data (*data source*) statis secara global.

Pada baris [4] sampai [69], variabel global `val songs` dideklarasikan yang diinisialisasi menggunakan fungsi `listOf(...)`. Karena menggunakan `listOf` daripada `mutableListOf`, koleksi data yang dihasilkan bersifat *immutable* (tidak dapat dimodifikasi, ditambah, atau dihapus setelah aplikasi berjalan), memastikan data tetap aman dan konsisten saat dirender di lapisan UI, misalnya menggunakan `RecyclerView`. Di dalam blok `listOf`, dilakukan pemanggilan konstruktor `Song(...)` secara berulang untuk melakukan

pemetaan dummy data. Setiap blok mengisi parameter sesuai kontrak data kelasnya secara berurutan, mengorganisasikan koleksi lagu.

strings.xml (eng)

Pada baris [1] sampai [17], elemen `<resources>` digunakan sebagai root tag yang secara mutlak mendefinisikan bahwa file ini berfungsi sebagai tempat penyimpanan value resource di dalam sistem Android.

Pada baris [2] sampai [8], dideklarasikan sekumpulan elemen `<string>` berukuran pendek. Elemen-elemen ini memetakan variabel ID spesifik seperti `app_name`, `label_album`, `btn_listen` dengan nilai antarmuka dalam bahasa Inggris.

Pada baris [10] sampai [16], dideklarasikan sekumpulan elemen `<string>` yang memuat paragraf panjang untuk mendeskripsikan informasi spesifik dari setiap lagu seperti `desc_i`, `desc_flash`, dan lainnya. Teks inilah yang direferensikan menggunakan ID, contohnya `R.string.desc_i` pada objek `SongData`.

strings.xml (in)

Pada baris [1] sampai [17], elemen `<resources>` kembali digunakan sebagai root tag. File ini secara khusus ditempatkan di dalam direktori `values-id` oleh sistem Android, terpisah dari `strings.xml` utama yang biasanya berada di folder `values` bawaan.

Pada baris [2] sampai [8], dideklarasikan elemen `<string>` untuk teks antarmuka berukuran pendek seperti tombol dan label yang telah diterjemahkan ke dalam Bahasa Indonesia. Atribut yang dibuat sama persis dengan yang ada di `strings.xml` versi bahasa Inggris.

Pada baris [10] sampai [16], dideklarasikan elemen `<string>` yang memuat paragraph terjemahan panjang untuk mendeskripsikan setiap lagu.

item_song.xml

Pada baris [2] sampai [10], komponen `MaterialCardView` digunakan sebagai root layout yang membungkus keseluruhan elemen kartu. Atribut `android:layout_width="match_parent"` memastikan kartu memenuhi lebar layar yang

tersedia. Penggunaan atribut `app:cardCornerRadius="16dp"` memberikan efek membulat pada keempat sudut kartu, `app:cardElevation="4dp"` memberikan efek bayangan tiga dimensi, dan `app:cardBackgroundColor="#FFC2D1"` menetapkan warna latar belakang statis *Pink Champagne* secara eksplisit.

Pada baris [12] sampai [15], `ConstraintLayout` disarangkan di dalam `MaterialCardView` sebagai tata letak utama pengatur posisi komponen. Pendekatan ini efisien karena `ConstraintLayout` memungkinkan antarmuka yang kompleks dan datar tanpa perlu menyarangkan banyak tata letak, sehingga performa rendering menjadi lebih optimal. Atribut `android:padding="12dp"` memberikan ruang kosong di sisi dalam agar elemen tidak menempel langsung pada tepi kartu.

Pada baris [17] sampai [24], komponen `ShapeableImageView` digunakan untuk menampilkan gambar cover album. Komponen diberi identitas `@+id/img_song` dengan dimensi statis 110dp x 110dp. Atribut `android:scaleType="centerCrop"` memastikan gambar memotong bagian tepinya agar proporsional dan tidak terdistorsi. Atribut `app:shapeAppearanceOverlay="@style/RoundedCornerImage"` adalah implementasi modern dari Material Design untuk memberikan modifikasi sudut membulat secara langsung pada gambar tanpa perlu membuat file `drawable` background terpisah.

Pada baris [26] sampai [36], `TextView` dengan ID `tv_title` bertugas merender teks judul lagu. Atribut `android:layout_width="0dp"` diterapkan. Posisi diikat di sebelah kanan gambar menggunakan `layout_constraintStart_toEndOf` dan di sebelah kiri teks tahun `layout_constraintEnd_toStartOf`, lebar 0dp akan memaksa teks judul untuk memanjang secara dinamis hanya pada sisa ruang kosong yang tersedia, mencegah teks bertumpuk dengan elemen lain jika judul terlalu panjang.

Pada baris [38] sampai [44], `TextView` dengan ID `tv_year` digunakan untuk mencetak tahun rilis. Komponen ini ada secara presisi di sudut kanan atas wadah menggunakan `app:layout_constraintEnd_toEndOf="parent"` dan `app:layout_constraintTop_toTopOf="parent"`.

Pada baris [45] sampai [53], TextView dengan ID tv_label_album berfungsi sebagai label teks statis. Penggunaan android:text="@string/label_album" mendemonstrasikan pemisahan nilai teks dari file tata letak. Elemen ini diposisikan tepat di bawah judul lagu.

Pada baris [55] sampai [63], TextView dengan ID tv_album_name menampilkan nilai dinamis dari nama album. Sama seperti judul, elemen ini menggunakan android:layout_width="0dp" agar teks mengisi ruang kosong yang tersisa dari sebelah kanan label album hingga ke batas tepi kanan *parent*.

Pada baris [65] sampai [75], komponen Button dengan ID btn_link mempresentasikan tombol aksi Listen. Desain warnanya menggunakan teks merah muda (#FB6F92) dan latar belakang putih (@color/white). Secara tata letak, tombol ini tidak terikat pada *parent*, tapi menyelaraskan posisi atas dan bawahnya (constraintTop_toTopOf dan constraintBottom_toBottomOf) langsung terhadap tombol detail (btn_detail), sehingga keduanya selalu sejajar secara horizontal.

Pada baris [77] sampai [86], komponen Button dengan ID btn_detail digunakan untuk navigasi ke halaman spesifik. Tombol ini menggunakan warna kebalikan dari tombol Listen. Posisinya dikunci di sudut kanan bawah kartu (constraintBottom_toBottomOf="parent" dan constraintEnd_toEndOf="parent"s) dan berada di bawah teks nama album. Tombol ini bertindak sebagai jangkar vertical utama untuk tombol Listen yang ada di sebelahnya.

fragment_home.xml

Pada baris [2] sampai [7], komponen LinearLayout digunakan sebagai root layout yang membungkus seluruh halaman aplikasi. Penggunaan atribut android:orientation="vertical" menginstruksikan sistem untuk menyusun semua anak elemen di dalamnya secara lurus dari atas ke bawah. Atribut android:background="#FFE5EC" memberikan warna latar belakang merah muda pada keseluruhan layar.

Pada baris [9] sampai [28], komponen `RelativeLayout` digunakan sebagai wadah untuk bilah navigasi atas. Pemilihan ini memungkinkan elemen-elemen di dalamnya diposisikan relatif terhadap layar atau elemen lain, tanpa perlu menggunakan nested layout tambahan.

Pada baris [13] sampai [18], `TextView` menampilkan judul. Nilai teks dipanggil dari sumber daya terjemahan menggunakan `@string/app_name`, diceetak dengan ukuran 22sp, dan bold.

Pada baris [20] sampai [27], `ImageButton` dengan ID `btn_language` digunakan sebagai tombol interaktif untuk mengganti bahasa. Atribut `android:layout_alignParentEnd="true"` memaksa tombol untuk merapat secara absolut ke ujung kanan layar. Atribut `app:tint="#FB6F92"` digunakan untuk mewarnai ulang ikon bawaan Android menjadi merah muda pekat.

Pada baris [30] sampai [37], komponen `RecyclerView` pertama dideklarasikan dengan ID `rv_highlight`. Komponen ini bertindak sama seperti `LazyRow` pada Jetpack Compose, yaitu untuk merender daftar lagu yang dapat digeser secara horizontal. Pemberian jarak tepi `android:paddingStart="8dp"` dan `android:paddingEnd="8dp"`, dipadukan dengan `android:clipToPadding="false"`. Atribut `clipToPadding="false"` memastikan bahwa saat pengguna menggeser daftar kartu lagu, kartu tersebut tetap dapat mengalir dan terlihat di area *padding* tanpa terpotong secara kasar, sehingga menciptakan carousel effect yang halus dan modern di pinggir layar.

Pada baris [39] sampai [43], komponen `RecyclerView` kedua dideklarasikan dengan ID `rv_songs`. Ini bertindak sama seperti `LazyColumn` pada Compose, yang bertugas merender daftar lagu vertical. Atribut `android:layout_width="match_parent"` dan `android:layout_height="match_parent"` memastikan komponen ini mengisi seluruh sisa ruang layar yang ada di bawah *header* dan *carousel* horizontal, dengan diberikan sedikit jarak atas menggunakan `android:layout_marginTop="8dp"`

fragment_detail.xml

Pada baris [2] sampai [6], komponen `androidx.core.widget.NestedScrollView` digunakan sebagai root layout. Karena panjang teks deskripsi lagu bisa bervariasi dan berpotensi melebihi tinggi layar perangkat, `NestedScrollView` memastikan seluruh konten di dalamnya dapat digulir ke bawah tanpa terpotong. Atribut `android:background="#FFE5EC"` memberikan warna latar belakang solid merah muda pada keseluruhan layar.

Pada baris [8] sampai [12], sebuah `LinearLayout` disarangkan tepat di dalam `NestedScrollView`. Hal ini diwajibkan karena `NestedScrollView` hanya diizinkan memiliki satu komponen anak. Atribut `android:orientation="vertical"` menginstruksikan sistem untuk menyusun semua elemen visual (gambar, teks, tombol) secara lurus dari atas ke bawah. Atribut `android:padding="16dp"` memberikan jarak aman di keempat sisi layar agar konten tidak menempel langsung pada pinggiran perangkat.

Pada baris [14] sampai [20], komponen `ShapeableImageView` digunakan untuk menampilkan gambar cover album. Dimensinya ditetapkan secara statis sebesar 320dp x 320dp. Atribut `android:layout_gravity="center"` memastikan gambar berada tepat di tengah layar secara horizontal. Sama seperti pada rancangan kartu lagu, atribut `android:scaleType="centerCrop"` dan `app:shapeAppearanceOverlay="@style/RoundedCornerImage"` digunakan untuk menjaga proporsi gambar sekaligus memberikan modifikasi sudut membulat yang modern.

Pada baris [22] sampai [28], `TextView` dengan ID `tv_detail_title` bertugas merender teks judul lagu. Tipografi dibuat menonjol sebagai hierarki visual tertinggi dengan ukuran huruf ekstra besar (28sp), dicetak tebal (`textStyle="bold"`), dan diberikan jarak atas `layout_marginTop="16dp"` untuk memisahkannya dari kover album.

Pada baris [30] sampai [44], terdapat dua `TextView` berurutan, yaitu `tv_detail_album` dan `tv_detail_year`, yang menampilkan nama album dan tahun rilis. Keduanya menggunakan ukuran huruf standar 16sp dengan warna teks hitam pekat menggunakan

@color/black dan jarak atas yang lebih rapat (4dp) untuk menandakan bahwa kedua informasi ini saling berkaitan erat.

Pada baris [46] sampai [53], TextView dengan ID tv_detail_desc dikhususkan untuk merender teks deskripsi atau informasi panjang terkait lagu. Pada komponen ini, disematkan atribut android:lineSpacingExtra="6dp". Menambahkan jarak spasi ekstra antarbaris teks secara signifikan meningkatkan tingkat readability pengguna saat membaca blok paragraf yang panjang di layar ponsel.

Pada baris [55] sampai [62], komponen Button dengan ID btn_back diletakkan di bagian paling bawah halaman sebagai aksi navigasi kembali. Atribut android:layout_width="match_parent" memaksa tombol jadi penuh mengisi lebar layar yang tersedia. Desain tombol menggunakan warna latar merah muda pekat (#FB6F92) dengan teks berwarna putih, serta memanggil nilai teks dari sumber daya terjemahan melalui @string/btn_back.

fragment_language.xml

Pada baris [1] sampai [7], komponen LinearLayout digunakan sebagai *root layout* yang mengendalikan seluruh halaman. Atribut android:orientation="vertical" menginstruksikan agar elemen di dalamnya disusun lurus dari atas ke bawah. Terdapat penggunaan atribut android:gravity="center". Atribut ini memaksa seluruh elemen anak di dalam tata letak ini untuk tertarik dan berkumpul tepat di poros tengah layar, baik secara vertikal maupun horizontal. Hal ini memberikan estetika centered layout khusus untuk menu pengaturan. Atribut android:padding="24dp" memberikan ruang bernapas yang cukup luas di keempat sisi layar, sementara android:background="#FFE5EC" memberikan warna latar belakang merah muda.

Pada baris [9] sampai [15], TextView bertugas menampilkan teks judul halaman. Teks ini dipanggil secara dinamis menggunakan @string/setting_language untuk mendukung multi language. Tipografinya dibuat menonjol dengan ukuran 24sp dan dicetak tebal (textStyle="bold"). Atribut android:layout_marginBottom="32dp"

disematkan untuk memberikan jarak spasi yang cukup renggang antara teks judul dengan deretan tombol di bawahnya.

Pada baris [17] sampai [23], komponen Button dengan ID `btn_english` dirender sebagai opsi untuk mengubah bahasa aplikasi ke bahasa Inggris. Atribut `android:layout_width="match_parent"` membuat tombol ini membentang penuh secara horizontal mengikuti batas *padding* 24dp dari *parent*. Desainnya diberikan warna latar belakang merah muda pekat (`#FB6F92`) melalui atribut `backgroundTint` dan warna teks putih murni (`@android:color/white`).

Pada baris [25] sampai [31], komponen Button kedua dideklarasikan dengan ID `btn_indonesia` sebagai opsi untuk Bahasa Indonesia. Desain, warna, dan proporsinya dibuat identik persis dengan tombol pertama untuk menjaga konsistensi antarmuka. Karena pengaruh orientasi vertikal pada root layout, tombol ini secara otomatis akan tersusun tepat di bawah tombol bahasa Inggris tanpa memerlukan pengaturan posisi tambahan.

activity_main.xml (XML)

Pada baris [2] sampai [6], komponen `androidx.constraintlayout.widget.ConstraintLayout` digunakan sebagai root layout. Penggunaan `ConstraintLayout` memastikan fondasi tata letak yang kuat dan responsif. Atribut `android:layout_width="match_parent"` dan `android:layout_height="match_parent"` digunakan agar tata letak ini agar penuh mengikuti ukuran layar perangkat.

Pada baris [8] sampai [14], dideklarasikan komponen `androidx.fragment.app.FragmentContainerView` dengan ID `nav_host_fragment`. Komponen ini bertindak sebagai kanvas utama (*container*) bagi penerapan `Navigation Component` di Android berbasis XML (setara dengan `NavHost` pada `Jetpack Compose`).

Pada baris [10], atribut `android:name="androidx.navigation.fragment.NavHostFragment"` menginstruksikan

sistem Android untuk menyuntikkan kelas NavHostFragment ke dalam wadah kosong ini. Pendekatan Single-Activity Architecture diterapkan, di mana aplikasi hanya memiliki satu Activity utama, sedangkan antarmuka visual (beranda, detail, pengaturan) diwujudkan dalam bentuk Fragment yang ditukar-tukar secara dinamis di dalam wadah ini.

Pada baris [13], atribut `app:defaultNavHost="true"` adalah pengaturan keamanan (fail-safe) yang vital. Pengaturan ini menghubungkan wadah navigasi dengan sistem navigasi fisik perangkat (system back button). Jika pengguna menekan tombol Back pada smartphone mereka, sistem akan secara otomatis memanggil layar sebelumnya dari pop back stack yang tersimpan di dalam container, bukan keluar dari aplikasi.

Pada baris [14], atribut `app:navGraph="@navigation/nav_graph"` bertugas menautkan wadah ini ke berkas peta rute navigasi (`nav_graph.xml`). Berkas tersebut berisi seluruh definisi rute layar beserta aksi perpindahannya, sehingga `FragmentContainerView` ini tahu persis fragmen mana yang harus dimuat pertama kali (start destination) dan bagaimana cara berpindah ke layar lainnya.

SongAdapter.kt (XML)

Pada baris [1] sampai [6], deklarasi package menunjukkan direktori file, diikuti oleh import untuk memanggil library fundamental pembentuk daftar antarmukan, yaitu `RecyclerView`, `LayoutInflater`, dan `ViewGroup`

Pada baris [8] sampai [12], dideklarasikan kelas `SongAdapter` yang mewarisi `RecyclerView.Adapter`. Kelas ini adalah controller yang menghubungkan data logika dengan komponen visual `RecyclerView`. Pada konstruktor kelas ini, ada tiga parameter, list yang merupakan sumber data bertipe `List<Song>`, serta `onDetailClick` dan `onLinkClick`. `onDetailClick` dan `onLinkClick` menerapkan konsep Higher-Order Functions. Dengan menjadikan click event sebagai parameter fungsi, adapter tidak perlu tahu tentang logika navigasi atau pembukaan browser. Ini memastikan kode tetap mematuhi prinsip Single Responsibility, di mana adapter murni hanya mengurus data visual, logika aksi diserahkan ke Fragment atau Activity yang memanggilnya.

Pada baris [9], atribut variabel list: `List<Song>` dimodifikasi dari `val` (imutabel) menjadi `var` (mutabel) agar data daftar lagu di dalam adapter dapat diganti secara dinamis tanpa perlu membuat instansi objek Adapter yang baru dari awal.

Pada baris [14], dideklarasikan kelas bersarang (inner class) `ViewHolder`. Kelas ini bertugas menyimpan cache referensi komponen UI dari satu kartu lagu. Karena menggunakan View Binding, konstruktor cukup menerima parameter binding, lalu meneruskan `binding.root` ke kelas induk `RecyclerView.ViewHolder`.

Pada baris [16] sampai [19], metode `onCreateViewHolder(...)` di-override. Sistem akan memanggil fungsi saat `RecyclerView` membutuhkan kartu visual baru karena kartu yang ada di memori tidak cukup untuk menutupi layar. Baris `ItemSongBinding.inflate(...)` bertugas menerjemahkan berkas `item_song.xml` menjadi objek visual menggunakan `LayoutInflater`, lalu mengembalikannya dalam bentuk objek `ViewHolder` yang baru.

Pada baris [21] sampai [35], metode `onBindViewHolder(...)` di-override. Di sini proses data binding terjadi. Fungsi ini dieksekusi setiap kali sebuah kartu bergulir masuk ke dalam layar. Variable `song` diekstrak dari koleksi list berdasarkan position saat itu. Dengan blok scope function `holder.binding.appy {...}`, proses penulisan menjadi lebih ringkas. Data dari objek `song` dimasukkan ke elemen UI secara presisi, `imageResId` disetel ke `imgSong` menggunakan `setImageResource`, sedangkan judul dan nama album dimasukkan ke `tvTitle` dan `tvAlbumName`. Di dalam blok ini, `setOnClickListener` didaftarkan pada tombol `btnDetail` dan `btnLink`. Saat tombol ditekan, tombol tersebut akan mengeksekusi parameter fungsi lambda (`onDetailClick` atau `onLinkClick`) yang telah dideklarasikan di awal kelas sambil membawa objek `song` yang relevan.

Pada baris [37] sampai [39], metode `getItemCount()` di-override untuk mengembalikan total Panjang data (`list.size`). Angka ini memberitahu `RecyclerView` berapa banyak total baris maksimum yang harus ia kelola untuk mengalkulasi proporsi scrollbar secara akurat.

Pada baris [41] sampai [44], ditambahkan metode operasional baru `updateData(newList: List<Song>)`. Fungsi ini bertugas mengganti daftar lagu lama dengan daftar yang baru didapat dari `ViewModel`, lalu memicu pembaruan tata letak antarmuka secara keseluruhan melalui perintah `notifyDataSetChanged()`.

HomeFragment.kt (XML)

Pada baris [1] sampai [18], deklarasi package menunjukkan direktori file, diikuti import library Android. Library yang dipanggil mencakup lifecycle antarmuka (`LayoutInflater`, `ViewGroup`, `View`), arsitektur Fragment, sistem navigasi modern (`findNavController`), pengelolaan tata letak `RecyclerView` (`LinearLayoutManager`, `PagerSnapHelper`), serta kelas `FragmentHomeBinding` yang dihasilkan secara otomatis oleh sistem View Binding.

Pada baris [20] sampai [22], dideklarasikan class `HomeFragment` yang mewarisi sifat dari kelas dasar `Fragment`. Di dalam kelas ini, terdapat implementasi pola backing property menggunakan `_binding` (yang bersifat nullable) dan `binding` (non-null) untuk mencegah memory leak.

Pada baris [24] dan [25], ditambahkan deklarasi variabel `viewModel` dan adapter menggunakan kata kunci `private lateinit var`. Pendekatan ini memungkinkan variabel dideklarasikan di awal dan menunda inisialisasi nilainya hingga siklus pembuatan layar tiba.

Pada baris [27] sampai [34], metode `onCreateView(...)` di-override. Fase lifecycle ini bertanggung jawab untuk menciptakan dan menggambar (inflate) antarmuka visual. Berkas `fragment_home.xml` diterjemahkan menjadi objek Kotlin melalui `FragmentHomeBinding.inflate(...)`, lalu akar tata letaknya (`binding.root`) dikembalikan ke sistem untuk ditampilkan ke layar.

Pada baris [36] sampai [88], metode `onViewCreated(...)` di-override. Fase ini dieksekusi tepat setelah antarmuka selesai digambar. Seluruh inisialisasi data dan logika interaksi didaftarkan.

Pada baris [39] dan [40], viewModel diinisialisasi menggunakan ViewModelProvider. Karena ViewModel pada proyek ini membutuhkan parameter, digunakanlah SongViewModelFactory untuk menyuntikkan parameter string "Data Master Music CAS" ke dalam konstruktor SongViewModel.

Pada baris [42] sampai [45], variabel adapter diinisialisasi dengan memberikan nilai awal berupa list kosong (emptyList()). Logika interaksi pada blok lambda onDetailClick dan onLinkClick dimodifikasi secara fundamental. Antarmuka tidak lagi menangani logika navigasi secara langsung, melainkan mengoperasikan data tersebut ke ViewModel melalui pemanggilan viewModel.onDetailClicked(song) dan viewModel.onIntentClicked(url). Hal ini memenuhi prinsip clean architecture di mana antarmuka hanya bertugas menerima aksi.

Pada baris [47] dan [53], pengelola tata letak (LayoutManager) untuk rvSongs (daftar vertikal) dan rvHighlight (daftar horizontal) dikonfigurasi. Keduanya menggunakan variabel adapter yang sama. Khusus untuk rvHighlight, kelas PagerSnapHelper() ditautkan agar perilaku scroll alami RecyclerView terkunci secara presisi ke satu kartu penuh setiap kali pengguna selesai menggeser layar, menduplikasi efek carousel.

Pada baris [55] sampai [57], fungsi OnClickListener didaftarkan pada btnLanguage untuk menangani navigasi rute penggantian bahasa.

Pada baris [59] sampai [87], diimplementasikan blok viewLifecycleOwner.lifecycleScope.launch yang dikombinasikan dengan repeatOnLifecycle(Lifecycle.State.STARTED). Blok ini bertugas untuk mengamati (observe) aliran data dari StateFlow di dalam ViewModel secara reaktif dan aman, memastikan pemantauan data hanya berjalan saat Fragment sedang aktif.

Pada baris [61] sampai [65], aliran data songList diamati. Setiap kali ViewModel mengirimkan daftar lagu baru, fungsi akan memanggil adapter.updateData(songs), yang secara otomatis akan merender ulang tampilan RecyclerView dengan data terkini.

Pada baris [67] sampai [75], aliran data navigationEvent diamati. Ketika pengguna menekan tombol detail, event ini akan menangkap objek lagu, membungkusnya ke

dalam Bundle, dan mengeksekusi `findNavController().navigate(...)`. Segera setelah navigasi berhasil, perintah `viewModel.onNavigationHandled()` dipanggil untuk mereset status event agar tidak terjadi navigasi ganda saat layar dirender ulang.

Pada baris [77] sampai [85], aliran data `intentEvent` diamati untuk menangani tombol Listen. Nilai URL yang ditangkap akan disisipkan ke dalam eksekusi `implicit intent` (`Intent.ACTION_VIEW`), memaksa Android membuka tautan tersebut di browser web pihak ketiga. event ini kemudian direset menggunakan `viewModel.onIntentHandled()`.

Pada baris [90] sampai [93], metode `onDestroyView()` di-override untuk secara eksplisit mengubah `_binding` kembali menjadi null. Ini adalah langkah mutlak pembersihan referensi untuk memastikan garbage collector dapat membersihkan memori visual yang sudah tidak digunakan, sehingga aplikasi terbebas dari kebocoran memori.

DetailFragment.kt (XML)

Pada baris [1] sampai [9], deklarasi package menunjukkan direktori file, diikuti dengan sekumpulan import untuk memanggil library dasar Android. Pustaka ini mencakup penanganan lifecycle antarmuka (`LayoutInflater`, `ViewGroup`, `View`), arsitektur `Fragment`, pengelola navigasi (`NavController`), serta kelas `FragmentDetailBinding` yang dihasilkan oleh sistem View Binding.

Pada baris [10] sampai [12], dideklarasikan class `DetailFragment` yang mewarisi kelas dasar `Fragment`. Sama halnya dengan `HomeFragment`, kelas ini menerapkan pola `backing property` dengan variabel `_binding` dan `binding`. Implementasi ini sangat krusial di dalam arsitektur `Fragment` untuk memisahkan referensi antarmuka visual dari objek kelas itu sendiri, hal ini untuk menghindari memory leak.

Pada baris [14] sampai [21], metode `onCreateView(...)` di-override untuk menangani fase awal penciptaan visual. Di dalam blok ini, `fragment_detail.xml` diterjemahkan menjadi objek visual melalui metode `FragmentDetailBinding.inflate(...)`, lalu akar tata letaknya (`binding.root`) dirender ke layar.

Pada baris [23] sampai [42], metode `onViewCreated(...)` di-override sebagai tempat utama untuk mengeksekusi logika setelah antarmuka selesai digambar.

Pada baris [26], ada proses penangkapan data. Argumen yang dikirimkan oleh `HomeFragment` sebelumnya melalui `Bundle` diekstrak menggunakan metode `getSerializable("song")`. Penggunaan operator `as?` (safe cast) di sini merupakan best practice Kotlin. Daripada melakukan force cast, `as?` akan secara aman mencoba mengonversi data menjadi tipe objek `Song`. Jika tipe datanya tidak cocok atau terjadi kesalahan, ia tidak akan membuat aplikasi crash, terhindar dari `ClassCastException`, melainkan hanya mengembalikan nilai `null`.

Pada baris [27] sampai [37], pengecekan null safety (`if(song != null)`) dilakukan sebelum memanipulasi antarmuka. Jika data lagu valid, data tersebut disuntikkan ke dalam komponen UI. Perintah `getString(R.string.label_album)` digunakan untuk memanggil teks statis dari resource (`strings.xml`), yang kemudian digabungkan dengan data dinamis menggunakan `"$labelAlbum ${song.albumName}"`. Pendekatan ini memastikan halaman detail tetap mendukung penuh fitur multi language yang diterapkan pada tingkat aplikasi.

Pada baris [39] sampai [41], fungsi `setOnClickListener` disematkan pada tombol `btnBack`. Di dalamnya, metode `findNavController().navigateUp()` dieksekusi. Berbeda dengan `navigate()`, perintah `navigateUp()` secara spesifik menginstruksikan sistem navigasi untuk membuang halaman detail ini dari atas pop back stack dan mengembalikan pengguna ke halaman sebelumnya secara aman tanpa menciptakan instansi halaman baru.

Pada baris [44] sampai [47], metode `onDestroyView` di-override. Ini adalah bagian yang melengkapi pola backing property sebelumnya. Fase `onDestroyView()` dipanggil ketiga tampilan visual (view hierarchy) fragment dihancurkan oleh sistem, namun instansi kelas `DetailFragment`-nya mungkin masih ada di memori. Dengan secara eksplisit menetapkan `_binding = null`, maka memutus referensi ke komponen UI yang sudah mati tersebut. Hal ini memungkinkan Garbage Collector (pembersih memori

Android) untuk segera membebaskan ruang memori, memastikan aplikasi berjalan optimal dan bebas dari memory leak.

LanguageFragment.kt (XML)

Pada baris [1] sampai [10], deklarasi package menunjukkan lokasi direktori file. Diikuti oleh sekumpulan import untuk memanggil pustaka arsitektur Android dasar (Bundle, LayoutInflater, View, ViewGroup, Fragment). Terdapat pemanggilan library `androidx.appcompat.app.AppCompatActivity` dan `androidx.core.os.LocaleListCompat`. Kehadiran dua library ini menandakan bahwa Fragment ini mengimplementasikan fitur multi language modern dari Android API, yang memungkinkan aplikasi untuk mengubah bahasa antarmuka secara tanpa mengubah pengaturan bahasa sistem perangkat secara keseluruhan

Pada baris [11] sampai [13], dideklarasikan class `LanguageFragment` yang mewarisi sifat `Fragment`. Di dalamnya, terdapat lagi pola backing property melalui variabel `_binding` dan `binding` menggunakan `FragmentLanguageBinding`. Konsistensi penerapan pola ini di seluruh Fragment menunjukkan best practice dalam mencegah memory leak.

Pada baris [15] sampai [22], metode `onCreateView(...)` di-override. Fase lifecycle ini bertugas merender visual `fragment_language.xml` menjadi objek antarmuka yang nyata menggunakan metode `FragmentLanguageBinding.inflate(...)`. Akar tata letaknya (`binding.root`) kemudian dikembalikan untuk digambar di layar.

Pada baris [24] sampai [33], metode `onViewCreated(...)` di-override untuk mendaftarkan logika interaksi pasca rendering visual. Fungsi `onCheckedChangeListener` disematkan pada dua tombol utama, yaitu `btnEnglish` dan `btnIndonesia`. Saat `btnEnglish` ditekan, fungsi akan mengeksekusi metode utilitas `local.setAppLocale("en")`. Sebaliknya, saat `btnIndonesia` ditekan, fungsi akan mengirimkan argument "id".

Pada baris [35] sampai [38], dideklarasikan fungsi khusus `private fun setAppLocale(languageTag: String)`. Fungsi ini adalah otak dari sistem multi language

dinamis di aplikasi ini. Pertama, perintah `LocaleListCompat.forLanguageTags(languageTag)` akan menangkap string kode bahasa (seperti “en” atau “id”) dan mengonversinya menjadi objek daftar lokal yang valid dan didukung oleh sistem Android. Kemudian, perintah `AppCompatActivity.setApplicationLocales(localeList)` dieksekusi. Perintah ini menginstruksikan aplikasi untuk segera memperbarui konfigurasi bahasanya dan memuat ulang nilai teks dari direktori resource yang sesuai. Pembaruan ini terjadi secara instan tanpa memerlukan restart dari aplikasi.

Pada baris [40] sampai [43], metode `onDestroyView()` di-override. Sama dengan halaman beranda dan detail, fase ini menghancurkan referensi kelas ke komponen visual dengan menyetel `_binding = null`. Ini adalah langkah pembersihan mutlak agar Garbage Collector dapat membersihkan memori visual yang sudah tidak digunakan saat pengguna berpindah ke layar lain.

MainActivity.kt (XML)

Pada baris [1] sampai [5], deklarasi package menunjukkan direktori file. Diikuti import yang memanggil library dasar Android, yaitu `Bundle` untuk menyimpan status antarmuka, `AppCompatActivity` sebagai fondasi aktivitas Android modern, dan `ActivityMainBinding`

Pada baris [7], dideklarasikan class `MainActivity` yang mewarisi sifat dari `AppCompatActivity`. Beda dengan `Jetpack Compose` yang menggunakan `ComponentActivity`, pewarisan `AppCompatActivity` di sini bersifat mutlak. Hal ini diwajibkan karena aplikasi menggunakan arsitektur `Navigation Component` dengan `NavHostFragment`, di mana dukungan terhadap transaksi `Fragment` tingkat lanjut dan kompatibilitas sistem operasi Android versi lama (`backward compatibility`) hanya disediakan secara penuh oleh kelas ini.

Pada baris [8], dideklarasikan variabel binding menggunakan kata kunci `private lateinit var`. Atribut `lateinit` adalah mekanisme keamanan memori di Kotlin. Karena elemen antarmuka visual belum ada saat kelas pertama kali dibuat di memori, `lateinit`

mengizinkan untuk mendeklarasikan variabelnya sekarang dan menunda pengisian nilainya hingga siklus pembuatan layar tiba, sehingga terhindar dari null-safety checks yang berlebihan di seluruh blok kode.

Pada baris [10] sampai [14], metode `onCreate(...)` di-override. Ini adalah lifecycle penting yang pertama kali dipanggil saat aplikasi diluncurkan.

Pada baris [11], perintah `super.onCreate(savedInstanceState)` dieksekusi untuk memastikan sistem Android menjalankan semua rutinitas persiapan bawaannya.

Pada baris [12], pada perintah `binding = ActivityMainBinding.inflate(layoutInflater)`, alat `LayoutInflater` bertugas menerjemahkan atau merender kode XML mentah yang ada di `activity_main.xml` menjadi objek visual di dalam memori, lalu menyimpannya ke dalam variabel `binding`.

Pada baris [13], perintah `setContentView(binding.root)` dieksekusi. Jika pada arsitektur lama perintah ini diisi dengan pemanggilan referensi `R.layout.activity_main`, pada pendekatan modern ini, cukup menyuplai `binding.root`. Properti `.root` mewakili tata letak akar tertinggi dari berkas XML (dalam hal ini `ConstraintLayout` yang membungkus `NavHostFragment`). Perintah ini memproyeksikan seluruh kanvas antarmuka tersebut ke layar perangkat fisik. Mulai dari sini, `MainActivity` akan bertindak murni sebagai bingkai kosong (host) yang menampung pertukaran layar `HomeFragment`, `DetailFragment`, dan `LanguageFragment`.

MusicApplication.kt (Compose)

Pada baris [1] sampai [4], deklarasi `package` menunjukkan lokasi direktori file, diikuti dengan pemanggilan `import` untuk library bawaan `android.app.Application` dan pustaka eksternal `timber.log.Timber`.

Pada baris [6] sampai [11], dideklarasikan class `MusicApplication` yang mewarisi sifat dari kelas dasar `Application`. Di dalam arsitektur Android, kelas `Application` adalah komponen dasar yang akan diinisialisasi dan dieksekusi pertama kali oleh sistem sebelum `Activity`, `Fragment`, atau layanan antarmuka apa pun dimuat ke dalam memori.

Pembuatan kelas ini bertujuan untuk menyimpan status global (global state) dan menginisialisasi pustaka pihak ketiga secara terpusat.

Pada baris [7] sampai [10], metode onCreate() di-override. Di dalam fase ini, perintah Timber.plant(Timber.DebugTree()) dieksekusi. Perintah ini menginstruksikan pustaka Timber untuk menanam pohon debugging di seluruh siklus hidup aplikasi. Dengan konfigurasi ini, aplikasi secara otomatis akan mencatat log pemantauan yang sangat berguna selama fase development, dengan fitur penamaan tag otomatis yang merujuk pada nama kelas tempat log tersebut dipanggil.

SongViewModel.kt (Compose)

Pada baris [1] sampai [8], deklarasi package menunjukkan letak direktori file untuk arsitektur Compose, diikuti dengan pemanggilan import untuk berbagai library penting. Library yang dipanggil mencakup komponen dasar MVVM (ViewModel, ViewModelProvider), library reactive programming Kotlinx Coroutines (MutableStateFlow, StateFlow, asStateFlow), serta pustaka pencatatan log Timber.

Pada baris [10], dideklarasikan class SongViewModel yang mewarisi sifat dari arsitektur ViewModel. Kelas ini memiliki sebuah konstruktor utama yang menerima parameter categoryName bertipe String. Hal ini membuktikan bahwa ViewModel dapat dikustomisasi dan menerima injeksi data saat pertama kali dibuat.

Pada baris [11] sampai [18], diimplementasikan konsep enkapsulasi status (State Encapsulation) menggunakan Flow. Terdapat tiga pasang variabel: daftar lagu, event navigasi, dan event tautan. Variabel dengan awalan garis bawah (seperti _songList) dideklarasikan sebagai MutableStateFlow dan bersifat private, jadi datanya hanya bisa diubah dari dalam ViewModel. Variabel tersebut kemudian diekspos sebagai StateFlow publik (seperti songList) melalui ekstensi .asStateFlow(). Pola ini memastikan antarmuka (UI Compose) hanya memiliki hak untuk membaca data dan merespons perubahannya, tanpa bisa mengubah isi datanya secara langsung.

Pada baris [20] sampai [22], terdapat blok `init`. Blok ini adalah konstruktor inisialisasi yang akan secara otomatis mengeksekusi metode `loadSongs()` sesaat setelah objek `SongViewModel` berhasil diciptakan di memori.

Pada baris [24] sampai [28], metode `loadSongs()` bertugas memuat pasokan data. Daftar lagu secara statis diambil dari `SongData.songs` dan disisipkan ke dalam nilai `_songList.value`. Selain itu, pustaka `Timber` digunakan melalui perintah `Timber.d(...)` untuk mencatat proses ke dalam tab `Logcat`, memberikan bukti tertulis bahwa data berhasil dimuat ke dalam `UI Compose` beserta jumlah itemnya.

Pada baris [30] sampai [38], dideklarasikan dua metode pengendali aksi, yaitu `onDetailClicked(song: Song)` dan `onIntentClicked(link: String)`. Kedua metode ini berfungsi sebagai jembatan penerima interaksi dari antarmuka pengguna. Sesuai prinsip `clean architecture`, saat pengguna menekan tombol di antarmuka `Compose`, antarmuka tidak melakukan perpindahan layar sendiri, tapi memanggil fungsi ini. Fungsi kemudian mencatat log interaksi (`Timber.d`) dan mengisi nilai objek ke dalam `_navigationEvent` atau `_intentEvent` agar dapat direspons oleh layar utama.

Pada baris [40] dan [41], dideklarasikan dua metode `handler`, yaitu `onNavigationHandled()` dan `onIntentHandled()`. Fungsi ini secara eksplisit mengembalikan nilai aliran data event menjadi `null`. Pembersihan state ini wajib dilakukan di `Jetpack Compose` segera setelah aksi selesai dieksekusi agar aplikasi tidak berulang kali melakukan navigasi saat terjadi `recomposition` (render ulang layar).

Pada baris [44] sampai [52], dideklarasikan class `SongViewModelFactory` yang mengimplementasikan antarmuka `ViewModelProvider.Factory`. Mengingat `SongViewModel` membutuhkan parameter `categoryName` di dalam konstruktornya, sistem `Android` tidak dapat membuat kelas ini secara otomatis. Oleh karena itu, `Factory` kustom ini dibuat.

Pada baris [46] sampai [49], fungsi `create(...)` memeriksa apakah kelas model yang diminta sesuai, lalu mengembalikan instansi `SongViewModel` yang disuntikkan

dengan parameter string tersebut. Jika kelas tidak cocok, baris 50 akan melempar pengecualian `IllegalArgumentException`.

MusicApplication.kt (XML)

Pada baris [1] sampai [4], deklarasi package menunjukkan lokasi direktori file, diikuti dengan pemanggilan import untuk library bawaan `android.app.Application` dan library eksternal `timber.log.Timber`.

Pada baris [6] sampai [11], dideklarasikan class `MusicApplication` yang mewarisi sifat dari kelas dasar `Application`. Di dalam arsitektur Android, kelas `Application` adalah komponen dasar yang akan diinisialisasi dan dieksekusi pertama kali oleh sistem sebelum `Activity`, `Fragment`, atau layanan antarmuka apa pun dimuat ke dalam memori. Tujuannya untuk menyimpan status global (global state) dan menginisialisasi library pihak ketiga secara terpusat.

Pada baris [7] sampai [10], metode `onCreate()` di-override. Di dalam fase ini, perintah `Timber.plant(Timber.DebugTree())` dieksekusi. Perintah ini menginstruksikan pustaka `Timber` untuk menanam pohon debugging di seluruh lifecycle aplikasi. Dengan konfigurasi ini, aplikasi secara otomatis akan mencatat log pemantauan, dengan fitur penamaan tag otomatis yang merujuk pada nama kelas tempat log tersebut dipanggil.

SongViewModel.kt (XML)

Pada baris [1] sampai [8], deklarasi package diikuti dengan pemanggilan import untuk komponen MVVM (`ViewModel`, `ViewModelProvider`), library `Coroutines Flow` (`MutableStateFlow`, `StateFlow`, `asStateFlow`), serta library logging `Timber`. Penggunaan menggantikan `LiveData` tradisional sebagai solusi reactive programming yang lebih modern di Kotlin.

Pada baris [10], dideklarasikan class `SongViewModel` yang mewarisi arsitektur dasar `ViewModel`. Konstruktor kelas ini dirancang untuk menerima satu parameter bernilai `String`, yaitu `categoryName`. Pendekatan ini menunjukkan bahwa `ViewModel` dapat dikonfigurasi secara spesifik berdasarkan parameter yang disuntikkan saat dibuat.

Pada baris [11] sampai [18], diimplementasikan konsep state encapsulation menggunakan pola backing property pada aliran data. Ada tiga pasang variabel reaktif: `_songList`, `_navigationEvent`, dan `_intentEvent`. Variabel yang diawali dengan underscore menggunakan tipe `MutableStateFlow` dan bersifat `private`, artinya nilai datanya hanya boleh diubah dari dalam `ViewModel` ini saja. Sementara itu, variabel tanpa underscore terekspose sebagai `StateFlow` publik (`read-only`) untuk diamati oleh `Fragment` atau `UI`.

Pada baris [20] sampai [28], terdapat blok inisialisasi `init` yang akan langsung menjalankan metode `loadSongs()` sesaat setelah objek `ViewModel` diciptakan. Di dalam metode `loadSongs()`, data daftar lagu secara statis diambil dari `SongData.songs` dan dimasukkan ke dalam `_songList.value`. Selain itu, metode `Timber.d(...)` dipanggil untuk merekam jejak bahwa proses pemuatan data telah dimulai dan berhasil dieksekusi.

Pada baris [30] sampai [39], dideklarasikan dua fungsi pengendali aksi, yaitu `onDetailClicked` dan `onIntentClicked`. Sesuai dengan prinsip `clean architecture`, antarmuka tidak lagi diizinkan untuk melakukan aksi navigasi atau peluncuran `intent` browser secara mandiri. Saat pengguna menekan tombol di antarmuka, antarmuka hanya bertugas memanggil fungsi. Fungsi ini kemudian akan mencatat log menggunakan `Timber.d` dan mengubah `value` pada `_navigationEvent` atau `_intentEvent`, yang nantinya akan diamati dan direspons oleh `UI`.

Pada baris [42] dan [43], dideklarasikan fungsi handler yaitu `onNavigationHandled()` dan `onIntentHandled()`. Fungsi ini bertugas untuk mengatur kembali nilai aliran data `event` menjadi `null`. Mekanisme ini merupakan `best practice` yang pneting di `StateFlow` agar aksi seperti perpindahan layar tidak tereksekusi berulang kali saat perangkat diputar atau layar dirender ulang (`recomposition`).

Pada baris [46] sampai [54], dideklarasikan `class SongViewModelFactory` yang mengimplementasikan antarmuka `ViewModelProvider.Factory`. Secara default, sistem operasi `Android` tidak tahu cara membuat instansi `ViewModel` yang memiliki

parameter di dalam konstruktornya. Oleh karena itu, factory wajib disediakan. Di dalam fungsi `create(...)`, dilakukan pengecekan keamanan tipe kelas (`isAssignableFrom`). Jika kelasnya cocok, factory ini akan mengembalikan instansi `SongViewModel` yang disuntikkan dengan parameter string tersebut dan jika tidak, sistem akan melemparkan pengecualian memori `IllegalArgumentException`.

Fungsi Debugger

Debugger adalah alat inspeksi kode interaktif yang berfungsi untuk menghentikan jalannya eksekusi program secara sementara (pause) pada baris kode tertentu yang disebut dengan Breakpoint. Fungsi utama dari Debugger adalah untuk membantu melacak alur eksekusi aplikasi secara real-time, memeriksa nilai variabel yang tersimpan di dalam memori saat itu juga, dan menemukan akar masalah (bug atau crash) secara presisi tanpa perlu menuliskan banyak kode logging seperti `Log.d` atau `Timber.d` secara manual dan berulang-ulang.

Cara Menggunakan Debugger

Untuk menggunakan fitur Debugger di Android Studio, berikut adalah langkah-langkah proseduralnya:

1. Menentukan titik breakpoint

Buka file Kotlin yang ingin diinspeksi. Klik pada area kosong di margin sebelah kiri nomor baris kode misalnya pada baris fungsi tombol ditekan sampai ada titik merah. Titik ini artinya aplikasi akan berhenti sejenak saat eksekusi mencapai baris tersebut.

2. Menjalankan mode debug

Gunakan tombol debug yang ikon kumbang/bug warna hijau.

3. Memicu aksi

Operasikan aplikasi di emulator atau perangkat fisik fisik seperti biasa. Lakukan aksi yang memicu baris kode yang diberi breakpoint.

4. Memeriksa variabel

Aplikasi akan otomatis berhenti sementara. Jendela Android Studio akan membuka panel Debugger di bagian bawah, dan melihat panel Variables untuk menginspeksi nilai dan status data yang sedang diproses oleh aplikasi saat itu.

Fitur Step Into, Step Over, dan Step Out

Setelah aplikasi terhenti di sebuah *Breakpoint*, pengembang dapat mengontrol alur eksekusi program baris demi baris menggunakan fitur navigasi berikut:

1. Step Into (F7)

Fungsinya untuk melangkah masuk lebih dalam ke pemanggilan sebuah metode atau fungsi. Jika baris kode yang sedang dieksekusi memuat pemanggilan fungsi lain, setelah menekan Step Into akan masuk ke baris pertama di dalam fungsi tersebut untuk melihat detail eksekusi internalnya.

2. Step Over (F8)

Fungsinya untuk mengeksekusi baris kode saat ini secara keseluruhan dan langsung melompat ke baris kode berikutnya di dalam file yang sama. Fitur ini digunakan ketika yakin bahwa fungsi yang ada di baris tersebut berjalan normal dan tidak perlu masuk ke dalamnya untuk memeriksa detailnya.

3. Step Out (Shift + F8)

Fungsinya untuk keluar dari dalam sebuah fungsi jika sebelumnya menggunakan Step Into dan Kembali ke baris kode asal yang memanggil fungsi tersebut. Menekan Step Out akan membiarkan sisa kode di dalam fungsi yang sedang ditinggalkan berjalan berjalan otomatis hingga selesai.

SOAL 2

Soal Praktikum:

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Pembahasan

Dalam arsitektur sistem operasi Android, kelas Application adalah base class yang berfungsi sebagai representasi dari keseluruhan lifecycle aplikasi itu sendiri. Ini adalah komponen pertama yang diinisialisasi oleh sistem Android saat proses aplikasi mulai dijalankan, sebelum Activity, Fragment, atau antarmuka visual apa pun dimuat ke dalam memori perangkat. Kelas ini akan terus hidup selama aplikasi tersebut belum ditutup sepenuhnya atau prosesnya belum dimatikan oleh sistem (RAM).

Fungsi utama kelas Application mencakup tiga aspek penting dalam arsitektur Android. Pertama, bertindak sebagai titik awal global initialization. Karena metode onCreate() di dalamnya dieksekusi paling pertama, kelas ini menjadi tempat yang sangat ideal dan direkomendasikan untuk menghidupkan library atau SDK pihak ketiga seperti *logging* Timber, analitik Firebase, atau dependency injection yang harus terus berjalan secara global selama aplikasi aktif. Kedua, kelas ini berfungsi sebagai pengelola lifecycle yang independen. Berbeda dengan Activity atau Fragment yang rentan dihancurkan dan dibuat ulang oleh sistem saat terjadi perubahan konfigurasi seperti saat rotasi layar HP, kelas Application tahan terhadap hal tersebut sehingga mampu memberikan stabilitas untuk menjaga proses utama tetap berjalan. Ketiga, kelas ini berperan sebagai penyedia ApplicationContext, yaitu konteks tingkat aplikasi yang sangat krusial ketika sebuah fungsi membutuhkan Context, seperti untuk mengakses database lokal atau resource sistem namun beroperasi di luar lifecycle antarmuka layar. Penggunaan konteks tingkat aplikasi ini secara efektif dapat mencegah terjadinya memory leak yang sering muncul jika developer sembarangan menggunakan Context yang berasal dari sebuah Activity.

TAUTAN GIT

<https://github.com/lisaryuna/Praktikum-Mobile>